Distributed Synthesis of Differentially Private Tabular Datasets

Yucheng Fu[*], *Tianyao Gu*[†], *Elaine Shi*[†], *Tianhao Wang*[*]
[*]*University of Virginia,* [†]*Carnegie Mellon University*
{zdp8uu, tianhao}@virginia.edu, tianyaog@andrew.cmu.edu, elaineshi@cmu.edu

Abstract

Differentially private synthetic data generation has emerged as a powerful tool for sharing data while protecting individuals' privacy. However, when the attributes of sensitive data are distributed across multiple entities such as hospitals, companies, or government agencies, accurately generating synthetic data becomes challenging. In particular, it is difficult to capture informative statistical correlations and use them to guide data synthesis without gathering the entire private dataset. In response to this challenge, we propose a secure multi-party computation protocol for differentially private tabular data synthesis in the distributed setting. Our protocol contains two new primitives. The first is a protocol that exploits distributed point functions to efficiently estimate two-way marginals (pairwise joint distributions of attributes) across vertically distributed data. The second is a protocol for generating noise via batched lookups in the cumulative distribution function table. As a concrete demonstration, we build a distributed version of AIM, a state-of-the-art DP data-synthesis algorithm. Our implementation achieves the same utility as its centralized version while reducing end-to-end runtime by orders of magnitude compared with prior work. For example, we can synthesize the "Adult" dataset in 24 minutes in a real-world WAN setting, whereas the existing protocol is estimated to take 57 days.

1 Introduction

Differentially private (DP) tabular data synthesis (TDS) aims to create a synthetic dataset that shares similar statistical properties as the original one [1, 2, 3, 4, 5]. This synthetic dataset can be used to support various downstream data analysis tasks, such as machine learning and complex SQL queries, without additional privacy loss. A common approach is to release DP multidimensional distributions over the ground-truth dataset to guide the post-processing generation algorithm. To illustrate the concrete synthesis workflow, we take AIM [3], the state-of-the-art centralized DP-TDS algorithm as an example. Given a tabular dataset D, it first computes all one-way and two-way marginals of D, where a one-way marginal $\mathbf{M}_i$ is the histogram over a single column i, and a two-way marginal $\mathbf{M}_{i,j}$ is the histogram on the cross-domain of column i and j. AIM proceeds iteratively until it consumes all the privacy budgets. At each step t, AIM finds a marginal from the original dataset that differs the most from the current synthetic dataset D_{t-1}. This marginal is selected using the exponential mechanism [6] to ensure DP. After adding noise to the selected marginal, the noisy marginal is released and fed into a graph-based generation algorithm, PGM [2], to tune the synthetic data and obtain a D_t. At last, the algorithm outputs D_T as the final synthetic dataset.

In practice, the dataset to be synthesized is often held by different data owners. Depending on how the dataset is partitioned, we have the *horizontally distributed* and *vertically distributed* settings. In the former setting, the data owners hold disjoint sets of rows in table [7, 8, 9, 10], where the marginal computation is identical to securely computing histograms [11, 12, 13]. The challenge arises in vertically distributed settings, where the columns of a dataset reside on different parties that do not trust each other, and we cannot compute the two-way marginals. For example, the patients' blood pressure and economic status may be held by two different organizations (hospitals and banks). In this case, if two organizations run the DP-TDS algorithm separately, the correlation between the attributes would be completely lost. To address this issue, the most relevant prior work for vertically distributed DP-TDS, CaPS [8], uses a generic MPC approach. To compute a two-way marginal, it must enumerate all possible value combinations and count their occurrences in the secret-shared dataset. To compute $\binom{m}{2}$ numbers of two-way marginal, such a generic MPC solution incurs $O_\lambda(m^2nu^2)$ communication and computation (when using $O_\lambda(\cdot)$, we treat the security parameter λ as a constant). To synthesize a small table with $n = 10^4$ rows and $m = 14$ columns, each with domain size $u = 20$, the protocol takes about 44.7 hours to compute all the two-way marginals. Since computing the two-way marginal is equivalent to computing joint histograms [8], we

can also use the Sort-count protocol for joint histograms [14]. Under the same data scale and setting, computing all the two-way marginals still takes 18.6 minutes (more details in Section 7.1). A recent work of Durak et al. [12] introduces an efficient protocol for joint histogram computation based on three-party shuffling. Unfortunately, under DP-TDS, the joint distribution must be kept as secret shares to support later selection (see Section 3 for details). However, the protocol in [12] supports only plain-text or DP histograms as outputs and thus cannot be applied to DP-TDS. Note that we can also use fully homomorphic encryption (FHE), but it works under different assumptions and is not concretely efficient.

Another efficiency bottleneck of using MPC lies in the secure noise generation. After obtaining the marginal secret shares, we still need a large number of secret-shared noise samples to perform subsequent randomization of DP. However, in MPC, it is challenging to efficiently generate noise from specific distributions (e.g., discrete Gaussian and Geometric distributions used in our protocol). Existing methods convert the uniformly random bits into bits from the Bernoulli distribution and compose these biased bits into the desired distribution [15, 16, 17, 18]. To ensure a negligible statistical distance to the desired distribution, the number of bit conversions and steps to convert a single bit increase with the security parameter λ. Therefore, these methods incur high communication and round complexity. We note that there are also other noise generation protocols (including the Gaussian noise generation in CaPS [8]), which are based on evaluating non-linear functions in MPC or direct polynomial approximation for the desired distribution [8, 19, 20, 21]. Although these methods might be more efficient, their privacy/security risks due to approximation remain unknown. For this reason, we exclude those protocols in our comparison.

In summary, to achieve efficient and accurate distributed DP-TDS, we encounter two challenges caused by the limitations of existing techniques: (1) large ($O_\lambda(m^2nu^2)$ asymptotically) communication and computation overhead in computing two-way marginals, and (2) the lack of an efficient noise sampling protocol with a precise DP guarantee.

1.1 Results and Contributions

Our work improves the marginal computation and noise generation. Afterward, we integrate them to obtain an efficient end-to-end protocol for data synthesis that preserves the utility of the centralized setting. We describe our contributions as follows.

Concrete Improvement on Key Components. We improve two fundamental building blocks of multi-party DP-TDS.

- **Two-way marginal computation.** We design a new protocol for two-way marginal computation in the distributed setting, which achieves substantial improvements in runtime. For example, on a real-world tabular dataset with $n = 10^4$ rows, $m = 14$ columns (distributed over 14 data owners), and domain size $u = 100$ per column, our protocol computes all two-way marginals within 133.63 seconds. This shows a $10^5\times$ improvement over the existing multi-party DP-TDS method [8] and a $33\times$ improvement over the alternative protocol for joint distribution computation [14].
- **Noise generation**. We propose a new protocol for sampling from discrete distributions such as the Geometric and discrete Gaussian, while preserving rigorous DP guarantees. Concretely, we can generate 2^{14} number of secret-shared discrete Gaussian samples with standard deviation 30 in 13.41 seconds, achieving $69\times$ improvement compared to the state-of-the-art protocol [16].

End-to-end Multi-party DP-TDS. We adapt the state-of-the-art centralized DP-TDS algorithm AIM [3] to the distributed setup, and benchmark its efficiency and utility. Our end-to-end protocol can synthesize the real-world dataset named "Adult" [22] (with $48,843$ records and a 6×10^{17} Cartesian product domain size) in 24.12 minutes, where the existing distributed method CaPS [8] is estimated to take 57 days [1]. Our synthetic dataset has the same utility as the centralized algorithm in complex downstream tasks such as machine learning and conditional SQL queries.

Implementation. The implementation of our protocols is currently available at https://doi.org/10.5281/zenodo.18218427.

1.2 Technical Overview

DPF-based Marginal Computation. Our protocol for marginal computation begins with a simple observation that a two-way marginal $\mathbf{M}_{a,b}$ can be expressed as the summation of n matrices.

$$\mathbf{M}_{a,b} = \sum_{i=1}^{n} \mathbf{v}^i_{a,b}$$

$$\text{where } \mathbf{v}^i_{a,b}[s][t] = \begin{cases} 1 & \text{if } s = \mathbf{Col}_a[i] \text{ and } t = \mathbf{Col}_b[i]; \\ 0 & \text{otherwise.} \end{cases}$$

Concretely, a matrix $\mathbf{v}^i_{a,b}$ represents the one-hot encoding of values in position ($\mathbf{Col}_a[i]$,$\mathbf{Col}_b[i]$). For example, suppose $\mathbf{Col}_a[i] = 1$ and $\mathbf{Col}_b[i] = 2$. Then, we have $\mathbf{v}^i_{a,b}[1][2] = 1$ and other elements in $\mathbf{v}_{a,b}$ are all 0. Now, we need all the matrices ($\mathbf{v}^i_{a,b}$, where $i \in [n]$) to be secret-shared among servers, such that these matrices can be locally summed up to the secret shares of $\mathbf{M}_{a,b}$. To obtain secret shares of $\mathbf{v}^i_{a,b}$, one possible way is to have client a and b directly sharing $\mathbf{v}^i_a$ and $\mathbf{v}^i_b$ respectively. Then, the servers can run the MPC protocol to compute the outer product of $\mathbf{v}^i_a$ and $\mathbf{v}^i_b$ (denoted by $\mathbf{v}^i_a \times \mathbf{v}^i_b$). Doing so incurs $O_\lambda(u)$ client-server communication for sharing and

[1] We measure the runtime of marginal computation in CaPS [8] for a single record and extrapolate to $n = 48,843$ records, as the runtime grows linearly with n.

Protocols / Asymptotic cost	Secret-shared output				Plain text output
	Generic MPC [8]	Sort-count [14]	FHE [23]	**Ours**	Precio [12]
Client computation	$O_\lambda(mn)$	$O_\lambda(mn)$	$O_\lambda(mn)$	$O_\lambda(mn\log u)$	$O_\lambda(mn)$
Client-server communication	$O_\lambda(mn)$	$O_\lambda(mn)$	$O_\lambda(mn)$	$O_\lambda(mn\log u)$	$O_\lambda(mn)$
Server-server communication	$O_\lambda(m^2nu^2)$	$O_\lambda(m^2(n+u^2)\log u)$	0	0	$O_\lambda\left(m^2n\right)$
Server computation	$O_\lambda(m^2nu^2)$	$O_\lambda(m^2(n+u^2)\log u)$	$O_\lambda(m^2nu^2)$	$O_\lambda(mnu)$	$O_\lambda\left(m^2n\right)$
Server rounds	$O(1)$	$O(\log u)$	0	0	$O(1)$

Table 1: Complexity comparison of protocols to compute all the $\binom{m}{2}$ numbers of two-way marginals. n denotes the number of rows, and m is the number of columns (also the number of clients), u is the domain size of each column, λ is the security parameter, which determines the size of finite field elements for secret sharing, the length of random seeds in DPF, and the key length in FHE. The computation and communication cost for clients refers to the total cost of all the clients instead of the per-client cost. The compared protocol includes: the prior work for distributed DP-TDS [8] and two alternative methods, Sort-count proposed in [14] for joint histogram estimation and using fully homomorphic encryption (FHE) [23] to compute the local products. We also consider a comparison with a recent work, Precio [12], that relies on secure shuffling. It shuffles two columns and reveals all the elements. The plain text two-way marginal can be locally computed by counting the shuffled elements. Precio cannot provide secret-shared output, thus it does not support further marginal selection.

$O_\lambda(u^2)$ server-server for multiplication, making the cost of this solution the same as naive linear scan [8].

In our protocol, we leverage the distributed point function [24] to compress each one-hot vector into keys of size $O_\lambda(\log u)$. The clients compute these keys and send them to the servers. Thus, each server will receive mn (equal to the size of the joint database) DPF keys prepared by clients. To get a two-way marginal $\mathbf{M}_{a,b}$, the servers locally evaluate every pair of DPF keys that belong to $\mathbf{Col}_a$ and $\mathbf{Col}_b$ to obtain the one-hot encodings $\langle \mathbf{v}_a^i \rangle$ and $\langle \mathbf{v}_b^i \rangle$ in replicated secret shares. With these replicated secret shares, the servers can compute the additive shares of $\mathbf{v}_a^i \times \mathbf{v}_b^i$ non-interactively, reducing server-to-server communication to 0. We refer the readers to Section 4 for more details. In Table 1, we compare the asymptotic cost with other alternative protocols.

Noise Generation with CDF Table Lookup. To achieve efficient noise generation in MPC, our first observation is that we can look up a pre-computed table that contains the cumulative distribution function (CDF). Given a uniformly random number, conducting a linear scan on the table indexed by the CDF of each possible value is sufficient to get the desired sample. However, naively repeating this process for d samples on a CDF table of length s incurs $O(ds)$ communication and $O(s)$ rounds. To reduce communication while keeping low round complexity, our second observation is to generate all random numbers at the beginning of the protocol and sort them with the CDF table. After that, we use a parallel segmented scan [25] through the concatenated table to map the random numbers to the noise sample. With p bits to represent the CDF value and by using three-party radix sort [26], we can achieve only $O((d+s)p)$ communication and $O(\log d + p)$ rounds. After configuring $s = s(\lambda, d)$ and $p = p(\lambda, d)$, we compare the asymptotic cost of our protocols with existing methods in Table 2. We refer the readers to Section 5 for more details of our protocol.

Protocols / Asymptotic cost	Communication	Rounds
ODO sampling [16, 15]	$O\left(d\phi \log \frac{\phi}{\varepsilon}\right)$	$O\left(d\phi \log \frac{\phi}{\varepsilon}\right)$
Ostack sampling [17, 18]	$O\left(d \log \phi \cdot \log \frac{\phi}{\varepsilon}\right)$	$O\left(d \log \phi \cdot \log \frac{\phi}{\varepsilon}\right)$
Ours (discrete Gaussian)	$O\left(d\phi + \frac{\phi^{3/2}}{\varepsilon}\right)$	$O\left(\log(d+\frac{1}{\varepsilon})+\phi\right)$
Ours (Geometric)	$O\left(d\phi + \frac{\phi^2}{\varepsilon}\right)$	$O\left(\log(d+\frac{1}{\varepsilon})+\phi\right)$

Table 2: Amortized cost of noise generation protocols. λ is the security parameter, ε is the privacy budget, d is the number of samples, and $\phi = \lambda + \log d$. Existing methods, ODO sampling and Ostack sampling, have the same asymptotic cost for both Gaussian and Geometric noise. The main difference is that sampling from a discrete Gaussian requires additional rejection sampling. The Ostack sampling [17, 18] outperforms the ODO sampling when λ is larger than about 256 [18], so in evaluation we use the ODO sampling as the baseline for comparison.

End-to-end Synthesis and Other Improvements. We combine the above two techniques to achieve end-to-end distributed DP-TDS. This end-to-end protocol first computes the secret shares of all the marginals. Then, it selects the worst approximated marginal in a way that satisfies DP and adds noise to the selected marginal. While prior work applies the multiplicative weights exponential mechanism [27] for marginal selection in DP-TDS, we instead adopt the report noisy max

mechanism with Geometric noise [17]. By doing so, we avoid computing a heavy and approximated exponential function in MPC [28] and reduce the selection process to the sampling of discrete noise. Furthermore, we introduce a preprocessing step called domain compression, which both improves the utility of the generated dataset and reduces the protocol's communication overhead. We refer the readers to Section 6 for more details.

2 Preliminaries

2.1 Notation

We use $\mathbb{G}$ to denote an Abelian group and λ to denote the security parameter. Let $[n]$ be an ordered set of integers $\{1,\ldots,n\}$ and $\mathbf{x}=\{x_1,\ldots,x_n\}$ be a vector. The elements in $\mathbf{x}$ with subset index $A\subseteq[n]$ is denoted by $\{x_i\}_{i\in A}$. Given a discrete distribution $\mathcal{V}$, we use $F_{\mathcal{V}}(\cdot)$ to represent its cumulative distribution function (CDF) and $f_{\mathcal{V}}(\cdot)$ to represent its probability mass function (PMF). We use $\mathcal{N}(\sigma)$ and $\mathcal{G}(\sigma)$ to denote the discrete Gaussian distribution and Geometric distribution with scale σ. In Appendix A, we define these two distributions.

2.2 Problem Statement and Threat Model

In this work, we consider the tabular dataset D as a matrix of size $n\times m$, with n being the number of records (rows) and m being the number of attributes (columns). We use $\mathcal{U}_a$ to denote the domain of attribute a, and assume that $\mathcal{U}_a$ has been encoded as integers $\{1,\ldots,u_a\}$, where $u_a=|\mathcal{U}_a|$. For two attribute domains $\mathcal{U}_a$ and $\mathcal{U}_b$, we use $\mathcal{U}_{a,b}=\mathcal{U}_a\times\mathcal{U}_b$ (cartesian product) to denote their cross-domain of size u_au_b.

Now, suppose that a dataset D with n rows is distributed over m clients, each of them holding a single column $\mathbf{Col}_i$. These clients hope to generate a new synthetic dataset D^{syn} without revealing their own columns to each other. Meanwhile, D^{syn} should satisfy DP. We assume three servers (which can be played by clients) to receive the secret-shared inputs from clients and run a differentially private protocol Π among the servers. After executing Π, servers obtain D^{syn}.

We now define the threat model for the protocol Π. Assuming a *semi-honest* adversary $\mathcal{A}$ who corrupts one single server and any number of clients, we write $\mathsf{Real}^{\Pi}_{\mathcal{A}}(1^\lambda,D)$ for the random variable corresponding to the view of $\mathcal{A}$ and the output of the honest servers in the execution of Π.

We also define an ideal world that allows us to characterize the security properties of Π. Suppose there is a trusted party evaluating an ideal functionality $\mathcal{F}$ that takes the input D of m clients and sends output synthetic dataset D^{syn} to all the servers, including the honest ones and the corrupted one. Let $\mathcal{S}$ be an ideal-world adversary (also called a *simulator*) who also corrupts one single server and any number of clients. The simulator $\mathcal{S}$ takes the dataset D^{syn} output by $\mathcal{F}$ and all the corrupted parties' input as inputs. It can output arbitrarily according to these views. We denote $\mathsf{Ideal}^{\mathcal{F}}_{\mathcal{S}}(1^\lambda,D)$ as the random variable corresponding to the output of $\mathcal{S}$ and the output of the honest parties.

Definition 2.1 (Secure Emulation)**.** *A protocol Π securely emulates a functionality $\mathcal{F}$ if for every polynomial-time real-world adversary $\mathcal{A}$ there is a polynomial-time ideal-world adversary $\mathcal{S}$ such that for every input D, there exists a negligible function $\mathsf{negl}(\cdot)$, such that for all λ, the ensembles $\{\mathsf{Real}^{\Pi}_{\mathcal{A}}(1^\lambda,D)\}_{\lambda,D}$ and $\{\mathsf{Ideal}^{\mathcal{F}}_{\mathcal{S}}(1^\lambda,D)\}_{\lambda,D}$ are computationally indistinguishable:*

$$\{\mathsf{Real}^{\Pi}_{\mathcal{A}}(1^\lambda,D)\}_{\lambda,D}\approx_c\{\mathsf{Ideal}^{\mathcal{F}}_{\mathcal{S}}(1^\lambda,D)\}_{\lambda,D}.$$

2.3 Differential Privacy

Let $\varepsilon\geq 0$ and $\delta\in[0,1]$, for a pair of random variables V and V', we write $V\approx_{\varepsilon,\delta}V'$ if the following holds for any set S,

$$\Pr[V\in S]\leq e^{\varepsilon}\cdot\Pr[V'\in S]+\delta$$
$$\Pr[V'\in S]\leq e^{\varepsilon}\cdot\Pr[V\in S]+\delta$$

We abuse notation: $\mathcal{F}(D)$ denotes the output $\mathcal{F}$ sends to everyone upon receiving $D=(\mathbf{Col}_1,\ldots,\mathbf{Col}_m)$ from clients $1,\ldots,m$ respectively.

Definition 2.2. *An ideal functionality $\mathcal{F}$ satisfies (ε,δ)-differential privacy ((ε,δ)-DP) if for any $|D\triangle D'|=1$,*

$$\mathcal{F}(D)\approx_{\varepsilon,\delta}\mathcal{F}(D'),$$

where $|D\triangle D'|=1$ denotes that D and D' are identical except for modifying one record.

We now define what it means for a protocol Π to be computationally differentially private, by adapting the notion of SIM$^+$-CDP [29].

Definition 2.3. *A protocol Π satisfies (ε,δ) computational differential privacy ((ε,δ)-CDP) if there is an (ε,δ)-DP ideal functionality $\mathcal{F}$ such that Π securely emulates $\mathcal{F}$.*

We also use the zero-concentrated differential privacy to prove privacy, which inherits the guarantee of centralized DP-TDS [3, 4, 2]. We provide a detailed definition and property of this notion in Appendix B.

2.4 Secret Sharing

In a three-party setting, we use $[\![x]\!]=([\![x]\!]_1,[\![x]\!]_2,[\![x]\!]_3)$ and $\langle x\rangle=(\langle x\rangle_1,\langle x\rangle_2,\langle x\rangle_3)$ to denote the additive and replicated secret shares of a private value x. Suppose x_1,x_2,x_3 are three random numbers in the group $\mathbb{G}$, such that $x_1+x_2+x_3=x$. In the additive secret-sharing, we simply have $[\![x]\!]_1=x_1$, $[\![x]\!]_2=x_2$, and $[\![x]\!]_3=x_3$. In the replicated secret sharing, we have $\langle x\rangle_1=(x_1,x_2)$, $\langle x\rangle_2=(x_2,x_3)$, and $\langle x\rangle_3=(x_3,x_1)$.

Secret Shares Conversion. We define a procedure $\langle x \rangle \leftarrow \text{Convert}(\llbracket x \rrbracket)$ that converts additive shares to replicated shares as: party P_i sends his share $\llbracket x \rrbracket_i = x_i$ to party P_{i-1}. By doing so, each party gets two different shares.

Local Multiplication of Replicated Shares. Given two replicated secret sharings $\langle x \rangle$ and $\langle y \rangle$ of private values x and y respectively, the parties can locally compute the additive shares of their product $z = x \cdot y$. Observing that $x = \sum_{i=1}^{3} x_i$ and $y = \sum_{i=1}^{3} y_i$, the product z can be expanded as:

$$z = (x_1 + x_2 + x_3)(y_1 + y_2 + y_3) = \sum_{i=1}^{3} \sum_{j=1}^{3} x_i y_j.$$

The parties can partition these nine terms such that each party P_i computes a partial sum z_i using only the shares it holds:

$$\begin{aligned} \llbracket z \rrbracket_1 &= x_1 y_1 + x_1 y_2 + x_2 y_1, \\ \llbracket z \rrbracket_2 &= x_2 y_2 + x_2 y_3 + x_3 y_2, \\ \llbracket z \rrbracket_3 &= x_3 y_3 + x_3 y_1 + x_1 y_3. \end{aligned}$$

Because $\sum_{i=1}^{3} \llbracket z \rrbracket_i = z$, the result $(\llbracket z \rrbracket_1, \llbracket z \rrbracket_2, \llbracket z \rrbracket_3)$ constitutes a valid additive secret sharing of $x \cdot y$. We also define the local outer product between vectors shared in replicated secret shares as $\llbracket \mathbf{z} \rrbracket = \langle \mathbf{x} \rangle \times \langle \mathbf{y} \rangle$.

Ideal Functionalities. We illustrate the necessary functionalities that we will invoke in a black-box manner within our protocol.

- $\mathcal{F}_{\mathsf{Arith}}$**.** The ideal functionality $\mathcal{F}_{\mathsf{Arith}}$ contains all the standard arithmetic MPC operations used in this paper, including: (1) $\langle r \rangle \leftarrow \mathcal{F}_{\mathsf{Arith}}.\mathsf{rand}(p)$, which generates and distributes secret shares of a uniformly random p-bit value r. (2) $\langle b \rangle \leftarrow \mathcal{F}_{\mathsf{Arith}}.\mathsf{gt}(\langle x \rangle, \langle y \rangle)$ returns a secret share $\langle b \rangle$, where $b = 1$ if $x > y$, else $b = 0$. (3) $\langle y \rangle \leftarrow \mathcal{F}_{\mathsf{Arith}}.\mathsf{abs}(\langle x \rangle)$, which returns $\langle y \rangle$, where $y = x$ if $x > 0$, else $y = -x$. We implement $\mathcal{F}_{\mathsf{Arith}}$ using the SPDZ framework [30, 31].

- $\mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{k} \rangle, \langle \mathbf{v} \rangle, p)$**.** It takes as input a vector of secret-shared keys $\langle \mathbf{k} \rangle$ and a vector of secret-shared payloads $\langle \mathbf{v} \rangle$, both of which have length n. It reconstructs the inputs, sorts $\mathbf{v}$ using the first p bits of $\mathbf{k}$ as the key in non-descending order. We instantiate it with the three-party radix sorting [26], with $O_\lambda(np)$ communication and $O(p)$ rounds.

- $\mathcal{F}_{\mathsf{Shuffle}}(\langle \mathbf{x} \rangle)$**.** It can obliviously shuffle the elements in vector $\langle \mathbf{x} \rangle$ in random order, which can be achieved within $O(n)$ communication and $O(1)$ rounds [26].

2.5 Distributed Point Function

We utilize the *distributed point function* (DPF) to estimate the joint distribution over any two attributes (columns), which serves as a key building block in our task.

Definition 2.4 (Point Function)**.** *Let $\mathbb{G}_{\mathsf{in}}$ be the input group and $\mathbb{G}_{\mathsf{out}}$ be the output group. A point function parametrized by the point $x \in \mathbb{G}_{\mathsf{in}}$ is a function that evaluates to $\beta \in \mathbb{G}_{\mathsf{out}}$ at $\alpha \in \mathbb{G}_{\mathsf{in}}$ and evaluates to 0 everywhere else. We use $f_{\alpha,\beta} : \mathbb{G}_{\mathsf{in}} \to \mathbb{G}_{\mathsf{out}}$ to denote a point function. By definition, $f_{\alpha,\beta}(\alpha) = \beta$ and $f_{\alpha,\beta}(x) = 0$ for $x \neq \alpha$.*

We now extend the definition of point function to the distributed point function (DPF). In particular, we define the three-party DPF for our setting. It is a technique to secret-share a point function $f_{\alpha,\beta}$ into three keys. Given key k^i, server P_i locally evaluates k^i given any $x \in \mathbb{G}_{\mathsf{in}}$ and outputs a replicated secret share $\langle f_{\alpha,\beta}(x) \rangle_i$ of $f_{\alpha,\beta}(x) \in \mathbb{G}_{\mathsf{out}}$.

Definition 2.5 (Three-party DPF)**.** *A three-party DPF parametrized by $\mathbb{G}_{\mathsf{in}}$ and $\mathbb{G}_{\mathsf{out}}$ is a pair of randomized algorithms $(\mathsf{Gen}, \mathsf{Eval})$ with the following syntax:*

- $\mathsf{Gen}(1^\lambda, \alpha, \beta)$: *Outputs three keys k^1, k^2, k^3.*
- $\mathsf{Eval}(1^\lambda, k^i, x)$: *Outputs a replicated secret share $\langle f_{\alpha,\beta}(x) \rangle_i$.*

Correctness. *Correctness ensures that for any λ and any $x \in \mathbb{G}_{\mathsf{in}}$, if $k^1, k^2, k^3 \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta)$, then*

$$\Pr\left[\langle s \rangle_i \leftarrow \mathsf{Eval}(1^\lambda, k^i, x)_{i \in [3]} : s = f_{\alpha,\beta}(x)\right] = 1.$$

Security. *A three-party DPF parametrized by $\mathbb{G}_{\mathsf{in}}$ and $\mathbb{G}_{\mathsf{out}}$ is secure if there exists a probabilistic polynomial-time simulator $\mathcal{S}$, such that for any $i \in [3]$, any $\mathbb{G}_{\mathsf{in}}$ with bit length that is a polynomial function in λ, and any $\alpha \in \mathbb{G}_{\mathsf{in}}$, the following distributions are computationally indistinguishable:*

$$\begin{aligned} &\left\{ \{k^i\}_{i \in [3]} \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta) : k^i \right\} \\ \approx_c &\left\{ \{\tilde{k}^i\}_{i \in [3]} \leftarrow \mathcal{S}(1^\lambda, \mathbb{G}_{\mathsf{in}}, \mathbb{G}_{\mathsf{out}}) : \tilde{k}^i \right\}. \end{aligned}$$

Security requires that any single key k_i can be simulated without knowledge of the special point α and secret value β.

A concrete construction of an N-party DPF (we instantiate it with $N = 3$) from a length-doubling PRG is given in [24]. Its key generation process $\mathsf{Gen}(1^\lambda, \alpha, \beta)$ requires $O(\log|\mathbb{G}_{\mathsf{in}}| + \log|\mathbb{G}_{\mathsf{out}}|/\lambda)$ PRG calls to generate a key of size $O(\lambda \log|\mathbb{G}_{\mathsf{in}}| + \log|\mathbb{G}_{\mathsf{out}}|)$ for each party. Its evaluation process $\mathsf{Eval}(1^\lambda, k^i, x)$ also needs $O(\log|\mathbb{G}_{\mathsf{in}}| + \log|\mathbb{G}_{\mathsf{out}}|/\lambda)$ PRG calls. In the case of full-domain evaluation, one can expand the entire Goldreich-Goldwasser-Micali Construction (GGM tree) [32] defined by DPF once and reuse intermediate seeds, instead of re-computing each path, which reduces the complexity to $O_\lambda(|\mathbb{G}_{\mathsf{in}}| \cdot \log|\mathbb{G}_{\mathsf{out}}|/\lambda)$ PRG calls.

3 Ideal Functionality for Data Generation

In this section, we illustrate how to generate a new dataset D^{syn} that follows the distribution of the original dataset D, in

Functionality $\mathcal{F}_{\mathsf{DPMarginal}}$

Parameters. Number of attributes m, privacy budgets ρ_1, ρ_2.
Inputs. Intermediate synthetic dataset D'.
Initialization (first call). Receive the real-data columns $\{\mathbf{Col}_i\}_{i\in[m]}$ and precompute all reference marginals:

$$\mathbf{M}_a \leftarrow \text{marginal of } \mathbf{Col}_a, \quad 1 \le a \le m,$$

$$\mathbf{M}_{a,b} \leftarrow \text{marginal of } (\mathbf{Col}_a, \mathbf{Col}_b), \quad 1 \le a < b \le m.$$

On each call.

- Compute candidate marginals from D':

$$\mathbf{M}'_a,\ 1 \le a \le m, \qquad \mathbf{M}'_{a,b},\ 1 \le a < b \le m.$$

- For $i \in \{a \mid 1 \le a \le m\} \cup \{a,b \mid 1 \le a < b \le m\}$, compute utility score $\tilde{q}_i \leftarrow \|\mathbf{M}_i - \mathbf{M}'_i\|_1 + \mathcal{G}\left(2\sqrt{\frac{2}{\rho_1}}\right)$.
- Let $i^* \leftarrow \arg\max_i \tilde{q}_i$.
- $\tilde{\mathbf{M}}_{i^*} \leftarrow \mathbf{M}_{i^*} + \mathcal{N}(2\sqrt{\frac{2}{\rho_2}}\mathbb{I})$.

Output. $i^*, \tilde{\mathbf{M}}_{i^*}$.

Figure 1: Ideal functionality $\mathcal{F}_{\mathsf{DPMarginal}}$ for releasing noisy marginal. $\mathcal{G}(\cdot)$ and $\mathcal{N}(\cdot)$ denote Geometric and discrete Gaussian noise respectively.

a way satisfying differential privacy. To formalize the security guarantee of our protocol later, we also define this process as an ideal functionality.

Marginal-based Synthesis. Existing centralized DP-TDS algorithms [1, 2, 3, 33] follow an iterative workflow. In each step $t \in [T]$, the algorithm makes black-box access to a subroutine that releases only one noisy marginal over D. This marginal is the one that is the most poorly approximated by the current synthetic dataset D_{t-1}, which will be used to guide an optimization algorithm, and outputs the next intermediate dataset D_t. The iterative procedure is designed to simplify the optimization at each step and to save the privacy budget, because we only release the marginal that is currently the most inaccurate. In other words, in each step, we focus on the marginal that needs improvement the most, so the optimization effort is always directed where it matters most. Note that the optimization algorithm is a post-processing step on the released noisy distribution, so it does not need to be executed inside the MPC protocol and accounts for only a small fraction of the overall runtime. Therefore, our work focuses on the secure implementation of releasing a noisy marginal.

An Adapted Ideal Functionality. In Figure 1, we define an ideal functionality $\mathcal{F}_{\mathsf{DPMarginal}}$ corresponding to the centralized procedure that releases a single DP marginal. It is adapted from the centralized multiplicative weights exponential mechanism [27]. This mechanism is further improved in the state-of-the-art DP-TDS mechanism named AIM [3]. We

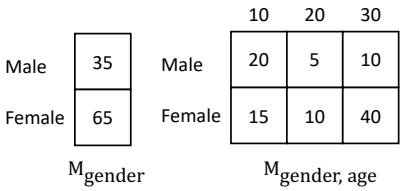

Figure 2: An example of one-way / two-way marginals. The column $\mathbf{Col}_{\mathsf{gender}}$ has domain {male, female} and the column $\mathbf{Col}_{\mathsf{age}}$ has domain {10,20,30}.

only present the ideal functionality for MWEM here because it is simple and can be easily upgraded to AIM by modifying the allocation of privacy budgets and adding the weight/penalty term in the utility score. Such modifications do not increase the burden in the runtime overhead for the algorithm. The original MWEM mechanism uses the exponential mechanism for marginal selection and adds continuous Gaussian noise to it. To facilitate efficient implementation in MPC, we make the following adaptations to $\mathcal{F}_{\mathsf{DPMarginal}}$ (we show that these modifications do not lead to utility loss in Section 7.3):

- **Selection mechanism.** Instead of using the exponential mechanism, we use the report noisy max to select two-way marginals. This is because implementing the exponential mechanism in MPC requires evaluating the secure exponential function, which is inefficient. With the report noisy max with Geometric noise, we reduce the procedure of DP selection to sampling from a discrete distribution in MPC.
- **Noise distribution.** We use discrete Gaussian noise, because it is much faster to generate in the MPC protocol and it can achieve a bounded privacy loss negligible in the security parameter (we analyze it in detail in Section 5.2).

This ideal functionality first computes all the one-way marginals and two-way marginals. In Figure 2, we provide an example of these marginals. With these marginals, it computes the distance between every two-way marginal in the ground-truth dataset D and the intermediate dataset D^{t-1} as $\|\mathbf{M}_i - \mathbf{M}_i^{t-1}\|_1$. It then selects a single marginal index i^* in a way satisfying DP and perturbs this marginal. We provide the DP guarantee of $\mathcal{F}_{\mathsf{DPMarginal}}$ and prove it in Appendix G.1

Lemma 1. *$\mathcal{F}_{\mathsf{DPMarginal}}$ satisfies $(\rho_1+\rho_2)$-zero concentrated differential privacy.*

We can also use Lemma 1 to quantify the privacy in (ε,δ)-DP, according to Proposition B.4.

Corollary 3.1. *$\mathcal{F}_{\mathsf{DPMarginal}}$ satisfies (ε,δ)-DP for all $\varepsilon \ge 0$ and $\delta = \min_{\alpha>1} \exp((\alpha-1)(\alpha\rho-\varepsilon)) \cdot \frac{1-\frac{1}{\alpha}}{\alpha-1}$.*

To securely emulate $\mathcal{F}_{\mathsf{DPMarginal}}$, we face two challenges, which motivate the design of two important components (Section 4 and Section 5):

- **Marginal computation.** Computing all the marginals in a way that preserves secrecy while remaining efficient. Especially, when the columns are distributed over different clients, it is hard to compute the two-way marginal, i.e., the joint histogram over two columns.
- **Noise generation within MPC.** Generating large amounts of discrete Gaussian and geometric noise inside an MPC, which is non-trivial in terms of efficiency.

4 Secure Marginal Computation

Given the overall ideal functionality, we now focus on the first component, secure marginal computation. In the next section, we will discuss another component, secure noise generation. We begin by describing two existing methods that can be used and discuss their drawback.

4.1 Competing Approaches

Baseline: CaPS with linear scan to the marginals [8]. A straightforward protocol to compute two-way marginals was proposed in the most relevant prior work. For each pair of secret-shared columns $(\langle \mathbf{Col}_a \rangle, \langle \mathbf{Col}_b \rangle)$, the protocol initializes a matrix $\langle \mathbf{M}_{a,b} \rangle$ of size $u_a u_b$ to store the counts of all the value combinations. Then, for $1 \le i \le n$, it conducts a linear scan on $\langle \mathbf{M}_{a,b} \rangle$ to find (j,k), such that $\langle \mathbf{Col}_a[i] \rangle = j$ and $\langle \mathbf{Col}_b[i] \rangle = k$, and then increments the corresponding count. This protocol can securely compute the secret-shared two-way marginals, as it hides which entry of the marginal matrix is updated. However, it is expensive to use in practice, because for every record $i \in [n]$, we need to check all $u_a u_b$ entries on the matrix, and it repeats for all $\binom{m}{2}$ attribute pairs. By treating the security parameter as a constant, the total communication/computation cost is $O_\lambda(m^2 n u^2)$ (for simplicity, we assume the columns have the same domain size u).

Alternative Solution: Sort-count protocol for joint histogram computation [14]. An alternative approach is using the recent protocol named Sort-count, which is designed for generic aggregation tasks [14]. As an adaptation to our setting, we can sort each column pair $(\langle \mathbf{Col}_a \rangle, \langle \mathbf{Col}_b \rangle)$ with $\langle \mathbf{Col}_a \rangle$ as the primary key and $\langle \mathbf{Col}_b \rangle$ as the secondary key. After sorting, identical value combinations appear consecutively. We can then perform a single pass over the sorted records and count how many times each combination appears. To ensure that all the value combinations exist during the linear scan, we need to additionally append all the possible value combinations from $\mathcal{U}_a \times \mathcal{U}_b$ before sorting. This method avoids scanning the full marginal matrix as in CaPS. Instead, the cost is dominated by the sorting itself. With existing three-party sorting protocols [26], each two-way marginal can be computed with $O_\lambda((n+u^2)\log u)$ communication, leading to a total cost of $O_\lambda(m^2(n+u^2)\log u)$ for all the attribute pairs.

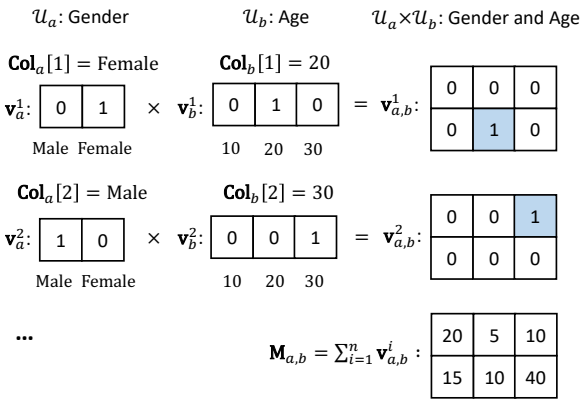

Figure 3: An example of computing two-way marginal using one-hot encoding. We assume $n = 100$. Let $\mathbf{Col}_a$ denote an attribute called "Gender", with domain $\mathcal{U}_a = \{\text{Male, Female}\}$, $\mathbf{Col}_b$ denote an attribute called "Age", with domain $\mathcal{U}_b = \{10, 20, 30\}$. We use "$\times$" to denote the outer product.

However, this protocol is still inefficient, as it must repeat $\binom{m}{2}$ times for all column combinations. In Appendix C, we describe the details of our adapted Sort-count protocol.

4.2 DPF-based Marginal Computation

We now describe our protocol for marginal computation based on three-party DPF.

Intuition. We illustrate our intuition on plaintext instead of secret shares held by servers. To compute a two-way marginal $\mathbf{M}_{a,b}$, we can expand each record $(\mathbf{Col}_a[i], \mathbf{Col}_b[i])$ into two one-hot encodings $(\mathbf{v_a}^i, \mathbf{v_b}^i)$. Afterwards, we can compute outer products to obtain a matrix as $\mathbf{v}_{a,b}^i = \mathbf{v_a}^i \times \mathbf{v_b}^i$, where i denotes the one-hot encoding of the i-th row. To get a two-way marginal $\mathbf{M}_{a,b}$, we can simply sum up all the n one-hot encoded matrix as $\mathbf{M}_{a,b} = \sum_{i \in [n]} \mathbf{v}_{a,b}^i$. We use an example to illustrate the process in Figure 3.

Protocol Instantiation via three-party DPF. Now we need servers to conduct the above process in a distributed manner, as described in $\Pi_{\mathsf{Marginals}}$ in Figure 4. At the beginning, each client $c \in [m]$ generates n key tuples on its private column $\mathbf{Col}_c$. Specifically, it decides the input domain size of DPF as $|\mathbb{G}_{\mathsf{in}}| = u_c$ to support the key evaluation over all the values in the attribute domain $\mathcal{U}_c$. It also sets the output domain size $|\mathbb{G}_{\mathsf{out}}| = O(n)$ to prevent potential overflow in the output marginals. With the specification of domains, client c sets the special point as $\mathbf{Col}_c[i]$ and runs the key generation algorithm to obtain a tuple of three keys, one for each server. After preparing the key, every client sends all the keys to the servers. In this communication step, each server receives $O(mn)$ DPF keys. Then, the servers can evaluate these DPF keys without any interaction. Specifically, for $c \in [m]$ and $i \in [n]$, each server conducts full domain evaluations on the

Protocol $\Pi_{\mathsf{Marginals}}$

Parameters. Number of records n, number of attributes m.
Prepare DPF Keys. Each client c computes DPF key tuple $\{k_{c,i}\} \leftarrow \mathsf{Gen}(1^\lambda, \mathbf{Col}_c[i], 1)$, for every record $\mathbf{Col}_c[i]$ in its column, and sends the keys in the tuple to P_1, P_2, and P_3.
Extract Marginals.

1. **DPF Key Evaluation.** For $c \in [m], i \in [n]$, servers evaluate their DPF key and get the one-hot encoding:
$$\langle \mathbf{v}_c^i \rangle \leftarrow \mathsf{FullEval}(1^\lambda, \{k_{c,i}\}).$$
2. **One-way Marginals.** For $1 \le a \le m$, compute:
$$\langle \mathbf{M}_a \rangle \leftarrow \sum_{i \in [n]} \langle \mathbf{v}_a^i \rangle.$$
3. **Two-way Marginals.** For $1 \le a < b \le m$, compute:
$$[\![\mathbf{M}_{a,b}]\!] \leftarrow \sum_{i \in [n]} (\langle \mathbf{v}_a^i \rangle \times \langle \mathbf{v}_b^i \rangle).$$

Outputs. Secret shares $\{\langle \mathbf{M}_a \rangle\}_{1 \le a \le m}$, $\{[\![\mathbf{M}_{a,b}]\!]\}_{1 \le a < b \le m}$.

Figure 4: The protocol $\Pi_{\mathsf{Marginals}}$ to compute all the two-way marginals. The outer product "$\times$" is taken locally on two vectors under replicated secret sharing, resulting in a vector under additive secret sharing.

i-th key from client c, which outputs its secret-shared one-hot encoding $\langle \mathbf{v}_c^i \rangle$. Now, to compute the secret shares of a two-way marginal $\mathbf{M}_{a,b}$, servers can locally compute the outer product of their shares: $[\![\mathbf{M}_{a,b}]\!] \leftarrow \langle \mathbf{v}_a^i \rangle \times \langle \mathbf{v}_b^i \rangle$ (see Section 2.4 for how the local multiplication work).

By plugging in $|\mathbb{G}_{\mathsf{in}}| = u$ and $|\mathbb{G}_{\mathsf{out}}| = O(n)$ into the cost of the underlying DPF scheme from Section 2.5 (w.l.o.g., we assume all attribute domains have the same size u and $\log n < \lambda$), we obtain the following theorem for the complexity.

Theorem 4.1. *The protocol $\Pi_{\mathsf{Marginals}}$, with DPF parameters $|\mathbb{G}_{\mathsf{in}}| = u$ and $|\mathbb{G}_{\mathsf{out}}| = O(n)$, requires $O_\lambda(n \log u)$ PRG calls per client, $O_\lambda(mnu)$ PRG calls per server, and $O_\lambda(mn \log u)$ bits of client-server communication.*

The communication cost of $\Pi_{\mathsf{Marginals}}$ only lies in the $3mn$ DPF keys sent from clients to servers. Thus, the total client-server communication of our protocol is $O_\lambda(mn \log u)$, and it does not incur server-server communication. As a comparison, CaPS [8] and Sort-count [14] both enumerate all $\binom{m}{2}$ column combinations, resulting in $O_\lambda(m^2 nu^2)$ and $O_\lambda(m^2 n \log u)$, respectively. Our protocol reduces the dependence on m from quadratic to linear, achieving a better asymptotic communication scaling in the number of columns.

Functionality $\mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}$

Parameters. Number of samples n, standard deviation σ.
Output. A secret-shared vector $\langle \eta \rangle = \{\langle \eta_i \rangle\}_{i \in [n]}$, where each η_i is sampled independently from the discrete Gaussian distribution $\mathcal{N}(\sigma)$.

Figure 5: The ideal functionality $\mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}$ for sampling discrete Gaussian noise and secret-sharing the result.

5 Noise Generation Protocol

In this section, we describe our protocol for sampling secret-shared noise from a discrete Gaussian / Geometric distribution, which serves as a key component for multi-party DP-TDS. We use a discrete Gaussian as an example to describe our construction, and the protocol for the Geometric distribution directly follows. We first define an ideal functionality $\mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}$ in Figure 5 and then discuss protocols to emulate it.

The best-known method for sampling discrete Gaussian noise is rejection sampling [16]. It first generates Bernoulli bits, composes them into Geometric samples, converts to two-sided discrete Laplace noise, and finally applies rejection sampling to get the discrete Gaussian sample. This bitwise computation is sequential and leads to high round complexity in the client-server model. An alternative is to use garbled circuits, which allow constant-round sampling but incur heavy communication due to large garbled tables, per-wire keys, and the cost of converting outputs back to arithmetic shares [34].

Intuition. While existing approaches based on bitwise sampling are suitable for generating a single noise sample, they do not have optimization for generating multiple samples. We observe that when multiple samples are needed, a more efficient strategy is possible. Our starting point is to use a simple table lookup instead of the complex generic MPC operations in prior protocols. Concretely, we pre-compute a table indexed by the cumulative distribution function (CDF), sample a uniformly random number in MPC, and compare it against the entries of the table. This baseline, however, still requires a full linear scan of the table for each noise sample. To reduce this overhead, we propose a batched lookup method that leverages secure sorting [26] and parallel segmented scan [25]. As a result, even when generating d samples, the table is still scanned only once.

5.1 Sampling with CDF Table

We begin with a basic CDF-based method as a stepping stone toward our optimized protocol, using the discrete Gaussian as the example. It works as follows to sample a secret-shared value $\langle \eta \rangle$, from the discrete Gaussian distribution $\mathcal{N}$.

1. **Pre-compute CDF table.** The servers construct a public

table $\mathcal{T}$ with two columns. The i-th element in the first column is a value $x_i \in [-s, s]$, and all the values are arranged in ascending order. The second column contains the values of CDF, represented by p-bits fixed-point number. Concretely, the i-th element is $f_{\mathcal{N}}(x_i)$.

2. **CDF table lookup.** All the servers jointly generate a secret-shared uniformly random number $[\![r]\!]$, $r \in [0,1]$. This can be done by requiring each server to secret-share a uniformly random number and sum them up. Then, the servers compare $[\![r]\!]$ with each CDF value in $\mathcal{T}$. With a linear scan over $\mathcal{T}$, we can find a value $\langle x_i \rangle$ as the output secret noise, such that $f_{\mathcal{N}}(x_i) \le r \le f_{\mathcal{N}}(x_{i+1})$.

The above naive method takes $O(s)$ communication and $O(s)$ rounds to scan the entire CDF table. When scaling to generate d samples, the communication becomes $O(ds)$ while the round complexity remains $O(s)$ because these samples can be generated with d trials in parallel. In the evaluation of [16], the bitwise sampling method still has better communication than this direct CDF table lookup method. Next, we demonstrate that by utilizing secure sorting and the parallel segmented scan, we can get a protocol that outperforms [16] in terms of both communication and round complexity.

5.2 Batched Lookup

Protocol Workflow. The protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ in Figure 6 implements CDF-based sampling with batch operation. We illustrate the main idea with the help of a small example in Table 3. In step 2 of Figure 6, the protocol begins by concatenating all the possible values in $[-s, s]$ with d elements (denoted as $\perp$) as placeholders. Correspondingly, we concatenate CDF values $\mathbf{d} = \{f(x_i)\}_{i=1}^{2s+1}$ with d uniformly random values $\{r_i\}_{i=1}^{d}$ to obtain a vector $[\![\mathbf{z}]\!]$. Now, instead of scanning the CDF table for each r_i, we observe that we can sort the combined vector $\mathbf{t}$ using $\mathbf{z}$ as the key to get $\mathbf{t}'$. Since the CDF is monotonic, there will be some random values placed between two values $f(x_i)$ and $f(x_{i+1})$ in the sorted vector $\mathbf{z}'$, and the dummy $\perp$ aligned with these random values will appear just after x_i in $\mathbf{t}'$. For example, in Table 3, after sorting, r_1 falls between $f(-1)$ and $f(0)$, so r_1 should be changed into -1. To do so, we apply a *propagation* operation in steps 4 and 5 of Figure 6. The goal of propagation is to replace each dummy with the closest valid sample value above it so that the random numbers $\{r_1, \ldots, r_d\}$ inherit the correct values from the CDF table. As shown in Table 3, after propagation, the three dummies are changed into -1, 0 and 1 respectively.

We instantiate such a propagation with the classical parallel segmented scan in [25]. It takes $O(\log(d+s))$ rounds to propagate the valid values instead of sequentially scanning the concatenated table with $O(d+s)$ rounds (see Appendix E for details). In the final step, we extract the d sampled values using a compaction procedure. Specifically, we sort $\mathbf{y}$ using $\mathbf{b}$ as a binary key, which is implemented by calling $\mathcal{F}_{\mathsf{Sort}}$ so that elements with $\mathbf{b}[i] = 0$ (i.e., new samples) are moved to the front. As shown in Table 3, the resulting vector $\mathbf{y}'$ contains the values $-1, 0, 1$ in the first three positions. A final secure shuffle removes any ordering correlation, yielding samples that can be used in downstream protocols.

Protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$

Parameters. Number of samples d, standard deviation σ, and security parameter λ.
Pre-compute. Set the truncation parameter s and CDF precision p (Appendix D) and construct a public table of size $2s+1$:

$$\mathbf{D} = \{x_i\}_{i=1}^{2s+1}, \quad \mathbf{F} = \{f(x_i)\}_{i=1}^{2s+1},$$

where $f(x_i)$ is the CDF of truncated discrete Gaussian $\mathcal{N}_s(\sigma)$, scaled to p-bit integers. Then secret-share both $\mathbf{D}$ and $\mathbf{F}$.
Compute.

1. Generate $\langle \mathbf{r} \rangle = \{r_i\}_{i=1}^{d}$, where $\langle r_i \rangle \leftarrow \mathcal{F}_{\mathsf{Arith}}.\mathsf{rand}(p)$.
2. Create a combined key vector and a sample vector aligned with the CDF table:
$$\langle \mathbf{t} \rangle = \langle \mathbf{D} \rangle \,\|\, \{\langle \perp \rangle\}_{i=1}^{n}, \quad \langle \mathbf{z} \rangle = \langle \mathbf{F} \rangle \,\|\, \langle \mathbf{r} \rangle.$$
3. Sort $\langle \mathbf{t} \rangle$ using $\langle \mathbf{z} \rangle$ as the key:
$$\langle \mathbf{t}' \rangle \leftarrow \mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{z} \rangle, \langle \mathbf{t} \rangle, p).$$
4. Initialize vector $\langle \mathbf{b} \rangle$ of size $2s+1+d$. For $i \in [2, 2s+1+d]$ in parallel, compute: $\langle \mathbf{b}[i] \rangle \leftarrow \mathcal{F}_{\mathsf{Arith}}.\mathsf{gt}(\langle \mathbf{t}'[i] \rangle, \perp)$.
5. $\langle \mathbf{y} \rangle \leftarrow \mathsf{segmented\_scan}(\langle \mathbf{b} \rangle, \langle \mathbf{t}' \rangle)$ (Appendix E).
6. Sort $\langle \mathbf{y} \rangle$ using $\langle \mathbf{b} \rangle$ as the binary key:
$$\langle \mathbf{y}' \rangle \leftarrow \mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{b} \rangle, \langle \mathbf{y} \rangle, 1).$$
7. Extract the first n entries and shuffle:
$$\langle \eta \rangle = \langle \mathbf{y}'[1:d] \rangle, \quad \langle \eta \rangle \leftarrow \mathcal{F}_{\mathsf{Shuffle}}(\langle \eta \rangle).$$

Outputs. A secret-shared vector $\langle \eta \rangle$ of d samples.

Figure 6: The batched inverse CDF protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ for discrete Gaussian sampling with secure sorting and shuffling. $\perp$ denotes a sufficiently small value, e.g., $\perp = -s-1$.

Parameter Setting of s and p. In Appendix D, we provide a detailed way to compute the truncation parameter s and CDF precision parameter p such that the d targeted samples output by the protocol have at most $2^{-\lambda}$ statistical distance to those from the "perfect" distribution $\mathcal{N}^d(\sigma)$. In summary, these parameters are configured as follows:

$$s = \lceil \sigma \cdot \sqrt{2 \ln 2 (\lambda + 2 + \log d)} \rceil,$$
$$p = \lceil \lambda + \log d + \log(2s+1) + \ln 2 + 1 \rceil.$$

Table 3: A small example of protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ with $s=2, n=3$. It begins with the concatenated vectors $\mathbf{z}$ and $\mathbf{t}$. After sorting, propagation, and compaction, we obtain $\mathbf{y}'$ storing three sampled values $\{-1,0,1\}$ from the targeted distribution. The vectors $\mathbf{t}'$ and $\mathbf{b}$ are shown for illustrative purposes and are not actually sorted.

Index	Before sorting		After sorting		Propagation		Compaction	
i	$\mathbf{z}$	$\mathbf{t}$	$\mathbf{z}'$	$\mathbf{t}'$	$\mathbf{b}$	$\mathbf{y}$	$\mathbf{b}$	$\mathbf{y}'$
1	$f(-2)$	-2	$f(-2)$	-2	1	-2	0	-1
2	$f(-1)$	-1	$f(-1)$	-1	1	-1	0	0
3	$f(0)$	0	r_1	$\perp$	0	0	0	1
4	$f(1)$	1	$f(0)$	0	1	0	1	-2
5	$f(2)$	2	r_2	$\perp$	0	0	1	-1
6	r_1	$\perp$	$f(1)$	1	1	1	1	0
7	r_2	$\perp$	r_3	$\perp$	0	0	1	1
8	r_3	$\perp$	$f(2)$	2	1	2	1	2

With the above configuration of protocol parameters, we can achieve an output noise distribution that has a statistical distance from "perfect" discrete Gaussian negligible in λ. We can also obtain the following security guarantee for $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$, proven in Appendix G.2.

Theorem 5.1. *Protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ securely emulates $\mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}$ in $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Sort}}, \mathcal{F}_{\mathsf{Shuffle}})$-hybrid model against semi-honest adversary.*

According to the configuration of parameters s and p, we also obtain the following theorem for the complexity of $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ (proven in Appendix G.3).

Theorem 5.2. *Let d be the number of samples, λ be the statistical security parameter, and σ be the scale parameter. Denote $\phi = \lambda + \log d$. Then $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ has $O\left(d\phi + \sigma\phi^{3/2}\right)$ communication complexity and $O\left(\log(d+\sigma)+\phi\right)$ round complexity.*

Implementing $\mathcal{F}^{\mathcal{G}}_{\mathsf{Noise}}$. The protocol $\Pi^{\mathcal{G}}_{\mathsf{Noise}}$ to sample noise from the Geometric distribution follows the same CDF table lookup as $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$. However, it has a different configuration of the truncation parameter s. According to the analysis in [17], truncating d Geometric noise into $[0,s]$ causes statistical distance $de^{-(\frac{s}{\sigma}-\ln d)}$. Similarly, by setting $de^{-(\frac{s}{\sigma}-\ln d)} = 2^{-\lambda}/2$ and then deriving s, we can set $s = \lceil \sigma \ln 2 \cdot \left(2\log d + \lambda + 1\right) \rceil$. As for the precision parameter, it matches that for discrete Gaussian: $p = \lceil \lambda + \log d + \log(2s+1) + \ln 2 + 1 \rceil$. In summary, we have $s = O(\sigma\phi)$ and $p = O(\phi)$. These give us a slightly different asymptotic cost for $\Pi^{\mathcal{G}}_{\mathsf{Noise}}$.

Corollary 5.3. *Let d be the number of samples, λ be the security parameter, and σ be the scale parameter. Denote $\phi = \lambda + \log d$. Then $\Pi^{\mathcal{G}}_{\mathsf{Noise}}$ has $O\left(d\phi + \sigma\phi^2\right)$ communication complexity and $O\left(\log(d+\sigma)+\phi\right)$ round complexity.*

Protocol $\Pi_{\mathsf{DPMarginal}}$

Parameters. Number of attributes m, privacy budgets ρ_1, ρ_2.
Inputs. Servers receive the intermediate synthetic dataset D_t.
Initialization (on first call). Clients send DPF keys to servers and servers call $\Pi_{\mathsf{Marginals}}$ to get $\{\langle \mathbf{M}_a \rangle\}_{1\le a\le m}$, $\{[\![\mathbf{M}_{a,b}]\!]\}_{1\le a<b\le m}$. Servers run Convert$(\{[\![\mathbf{M}_{a,b}]\!]\}_{1\le a<b\le m})$ to get replicated shares $\{\langle \mathbf{M}_{a,b} \rangle\}_{1\le a<b\le m}$.
On each call.

1. Compute candidate marginals from D':

$$\mathbf{M}'_a,\ 1 \le a \le m, \qquad \mathbf{M}'_{a,b},\ 1 \le a < b \le m.$$

2. For $i \in \{a \mid 1 \le a \le m\} \cup \{a,b \mid 1 \le a < b \le m\}$,
   - Compute score:

$$\langle q_i \rangle \leftarrow \sum_{j=1}^{|\langle \mathbf{M}_i \rangle|} \mathcal{F}_{\mathsf{Arith}}.\mathsf{abs}(\langle \mathbf{M}_i[j] \rangle - \mathbf{M}'_i[j]).$$

   - Add geometric noise: $\langle \tilde{q}_i \rangle \leftarrow \langle q_i \rangle + \mathcal{F}^{\mathcal{G}}_{\mathsf{Noise}}\left(1, 2\sqrt{\frac{2}{\rho_1}}\right)$.

   Select $i^* \leftarrow \arg\max \langle \tilde{q}_i \rangle$ with linear scan and reveal i^*.
3. $\langle \tilde{\mathbf{M}}_{i^*} \rangle \leftarrow \langle \mathbf{M}_{i^*} \rangle + \mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}\left(|\langle \mathbf{M}_{i^*} \rangle|, 2\sqrt{\frac{2}{\rho_2}}\right)$.

Output. Selected index i^* and the noisy marginal $\tilde{\mathbf{M}}_{i^*}$.

Figure 7: Protocol $\Pi_{\mathsf{DPMarginal}}$ that securely implements the computation, selection, and release of marginals.

6 Putting Things Together

6.1 Protocol for DP Marginal

In Figure 7, we present our protocol $\Pi_{\mathsf{DPMarginal}}$ that securely implements $\mathcal{F}_{\mathsf{DPMarginal}}$. The clients first prepare the DPF keys for $\Pi_{\mathsf{Marginals}}$ and then send them to the servers. After the execution of $\Pi_{\mathsf{Marginals}}$, the servers hold the secret shares of all the two-way marginals. Later, each time $\Pi_{\mathsf{DPMarginal}}$ is called, the servers receive the D' as the public input and compute a set of public two-way marginals $\{\mathbf{M}'_{a,b}\}_{1\le a<b\le m}$. We note that this can be done by only one server locally, as all the operations on D' are post-processing. The servers then jointly run a general MPC process to output the selected two-way marginal and its index. In this protocol, all the secret shares of noise are returned by $\mathcal{F}_{\mathsf{Noise}}$. The security of $\Pi_{\mathsf{DPMarginal}}$ is described as follows and proven in Appendix G.4.

Theorem 6.1. *$\Pi_{\mathsf{DPMarginal}}$ securely emulates $\mathcal{F}_{\mathsf{DPMarginal}}$ in $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Noise}})$-hybrid model against semi-honest adversary controlling one single server.*

According to Definition 2.3 and Corollary 3.1, the above theorem implies computational differential privacy.

Corollary 6.2. *$\Pi_{\mathsf{DPMarginal}}$ satisfies (ε, δ)-CDP.*

Algorithm 1 Distributed SDG

Input: Columns held by clients $\mathbf{Col}_1, \ldots, \mathbf{Col}_m$ and their domains $\mathcal{U}_1, \ldots, \mathcal{U}_m$, privacy budgets ρ_0, ρ_1, ρ_2, number of rounds T.

Output: The synthetic dataset D_T.

1: **for** $i \leftarrow 1$ to m **do**
2: $\quad \hat{\mathbf{Col}}_i, \hat{\mathcal{U}}_i \leftarrow \mathsf{data\_preprocessing}(\mathbf{Col}_i, \rho_0, \mathcal{U}_i)$
3: **end for**
4: Randomly initiate dataset D_0.
5: **for** $t \leftarrow 1$ to T **do**
6: $\quad \tilde{\mathbf{M}}_t \leftarrow \Pi_{\mathsf{DPMarginal}}(D_{t-1}; m, \frac{\rho_1}{T}, \frac{\rho_2}{T})$.
7: $\quad D_t \leftarrow \mathsf{generate}(\tilde{\mathbf{M}}_t; D_{t-1})$.
8: **end for**
9: **return** D_T.

In Appendix F, we discuss how to extend our protocol to be secure against malicious adversaries.

6.2 Compressing Domain Size

The actual cost of our protocol depends heavily on the size of each two-way marginal. In particular, when one attribute has a large domain, the entire protocol becomes costly. To address this issue, we adapt the domain compression algorithm in existing DP data synthesis algorithms [4, 3, 35] and design a local preprocessing algorithm $\mathsf{data\_preprocessing}()$ for clients. The high-level idea of $\mathsf{data\_preprocessing}()$ is to merge the categories with small count into a new category named "rare". Each client computes the one-way marginal for a column, adds discrete Gaussian noise with standard deviation σ, and marks categories with noisy counts below a threshold (e.g., 3σ) as rare. These categories are merged into a new placeholder bucket "rare", and the input column is updated accordingly. By doing so, we can obtain a new domain size $\hat{u}_i$ for each attribute $\mathbf{Col}_i$, such that $\hat{u}_i = O(n)$ (this is the worst case where the data distribution is very uniform). A detailed algorithm of this process is in Appendix H.

6.3 End-to-end Synthesis

In Algorithm 1, we describe an end-to-end distributed DP-TDS that only involves the clients. At the beginning of distributed DP-TDS, every client locally calls the $\mathsf{data\_preprocessing}$ algorithm to encode the values in its attribute and compress the domain size if necessary. Then, the data domain of each attribute becomes $\hat{\mathcal{U}}_i$. After locally computing the DPF keys and sending them to servers, the clients can go offline. In each step $1 \le t \le T$, the servers run $\Pi_{\mathsf{DPMarginal}}$ and output the selected DP marginal $\tilde{\mathbf{M}}_t$ to refine the synthetic dataset D_{t-1}. Note that we evenly split the privacy budgets ρ_1 and ρ_2 for simplicity of expression. When instantiating a specific DP-TDS, such as AIM [3], we can compute the privacy budgets in each round based on its budget allocation strategy, or even dynamically allocate budget in each round until all the budgets are consumed.

7 Experimental Evaluation

Our evaluations are conducted on a c5a.8xlarge AWS instance with 64 GB RAM and 32 vCPUs. We simulate the clients and three servers in the WAN setting, with bandwidth up to 0.5 Gbit/s and a latency of 50 ms.

7.1 Two-way Marginal Computation

In this section, we evaluate the efficiency of our two-way marginal computation protocol (Section 4). We implement the three-party DPF based on Google's code for incremental DPF [2], and use it to build our protocol for marginal computation. We also implement two baselines for comparison. The first one is the existing work for DP-TDS, named CaPS [8], we use their code [3] implemented in the MP-SPDZ framework [31] to reproduce the results. The second one is the Sort-count protocol in [14]. We also implement this protocol for two-way marginal computation using the MP-SPDZ framework. All the protocols are executed in arithmetic replicated shares over a 64-bit prime field (which is also the output secret shares of three-party DPF).

The experiments are conducted on generated datasets of different sizes. We fix the number of columns $m = 14$ (which matches the number of columns in "Adult" dataset), vary the number of rows $n \in [10^3, 10^4]$, and the domain size of each attribute as $u \in [20, 200]$. We use the global communication (including both client-server and server-server communication) and wall-clock running time as the metric to measure efficiency. Since the baseline CaPS has a high asymptotic cost, we evaluate it on a single two-way marginal and estimate its cost for all the $\binom{m}{2} = 91$ two-way marginals.

As shown in Figure 8, our method achieves at least a four-order-of-magnitude improvement in communication and a five-order-of-magnitude improvement in running time compared with the existing protocol, CaPS. Compared with the alternative method Sort-count, we also make $122\times \sim 1218\times$ improvement in communication and $30\times \sim 290\times$ improvement in running time. The reason for such a large improvement is due to the lightweight property of our protocol design. Our protocol only requires a single round of client-server communication to send the DPF keys of size $O_\lambda(mn \log u)$ and does not require any server-server communication. In our parameter setting, the total size of these keys is only 19.17 MB $\sim$ 213.66 MB. Both baseline methods require substantial inter-server interaction to perform generic MPC operations, thereby incurring high communication costs.

[2] https://github.com/google/distributed_point_functions
[3] https://github.com/sikhapentyala/MPC_SDG/tree/icml [8]

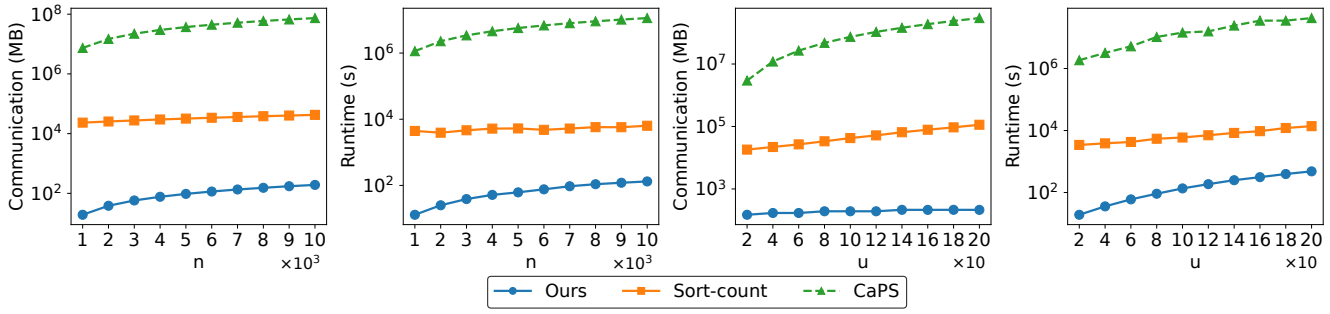

Figure 8: Communication/wall-clock running time to compute two-way marginals. When varying the number of rows n, we fix the domain size of each attribute $u = 100$. When varying the domain size of each attribute, we fix the number of rows $n = 10,000$.

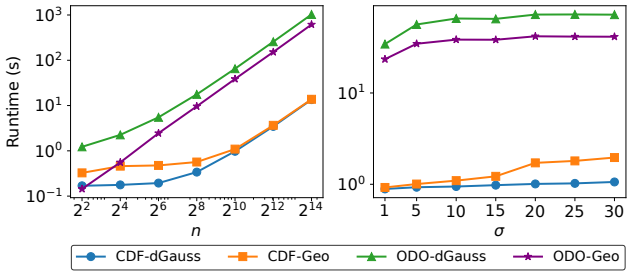

Figure 9: Runtime to sample discrete Gaussian samples and Geometric samples under different numbers of noise samples n and scales σ. The left figure fixes $\sigma = 10$, and the right one fixes $n = 2^{10}$.

Overhead	CDF-dGauss	CDF-Geo	ODO-dGauss	ODO-Geo
Time (s)	13.41	14.05	932.65	535.49
Comm (MB)	1998.6	2128.32	76.10	44.69
Rounds	4,234	4,342	12,017,375	7,159,845

Table 4: Detailed time, communication, rounds comparison across different sampling protocols, with $n = 2^{14}$ and $\sigma = 30$.

7.2 Noise Generation

In this section, we evaluate the efficiency of our CDF-based sampling protocol (Section 5) for Geometric noise and discrete Gaussian noise. We also include the ODO-based sampling for geometric [15] noise and discrete Gaussian [16] noise for comparison. All the protocols are implemented in the MP-SPDZ framework. Since the ODO-based protocols contain a large number of bitwise computations, we run these two protocols in the binary replicated secret shares (over the field $\mathbb{Z}_2$) and our CDF-based protocols in a 64-bit prime field. To align the statistical security guarantee, we set $\lambda = 32$ for all the noise generation protocols, which is equivalent to adding 2^{-32} to the failure probability δ in differential privacy. We show the evaluation result in Figure 9.

We provide a detailed comparison of time, communication, and rounds in Table 4. We note that our protocol does not outperform the ODO-based sampling protocols in terms of communication cost. However, our protocol significantly outperforms ODO-based protocols in terms of the number of rounds. Since latency is a critical factor when running protocols over real-world networks, reducing the round complexity is highly important. Let R be the number of rounds, L the network latency per round, C the total communication, and B the available bandwidth. In practice, the total running time can be approximated by $T_{\text{total}} \approx R \cdot L + \frac{C}{B}$. This relationship explains why our protocol can significantly outperform ODO-based protocols in actual executions. We also note that it is possible to run ODO-based sampling using constant-round protocols such as BMR or Yao's garbled circuits. These approaches involve additional cryptographic operations for encryption and decryption, as well as the transmission of wire labels that are much larger than secret shares. Using these protocols for ODO-based sampling often results in higher communication costs and longer running times than our CDF-based sampling. For example, running ODO-based sampling for discrete Gaussian noise with replicated-BMR, under the same parameters as in Table 4, took 9 hours and incurred 33GB of communication in our evaluation. Thus, in practical settings, our protocol achieves the best overall performance.

7.3 End-to-end Synthesis

We implement $\Pi_{\mathsf{DPMarginal}}$ in Figure 7 by combining the three-party DPF and the noise generation protocols to evaluate the end-to-end performance of Distributed DP-TDS (Algorithm 1). For the local computations `data_preprocessing()` and `generate()`, we use the code in [36]. Our implementation follows the process of the SOTA DP-TDS algorithm AIM [3], except that we replaced the code for the marginal computation, selection, and measurement with our implementation for $\Pi_{\mathsf{DPMarginal}}$. We conduct the data synthesis task on the UCI Adult dataset [22] with 48,843 rows of records, 14 attributes, and $2 \sim 100$ attribute domain sizes.

End-to-end Efficiency. Table 5 shows the efficiency of end-to-end distributed AIM. The synthesis algorithm calls $\Pi_{\mathsf{DPMarginal}}$ for 35 times until it consumes all the privacy

Table 5: End-to-end synthesis breakdown with $\varepsilon = 1$.

Overhead	Total	Marginals	ABS	Noise	Argmax	Generate
Time (min)	24.12	2.37	8.21	6.27	0.08	7.19
Comm (MB)	44946.51	542.28	21289.89	12.45	787.87	—
Rounds	1,037,871	1	857,710	132,805	47,355	—

budget. We observe that our protocol takes only 2.37 minutes to compute the secret shares for all marginals. On each call, $\Pi_{\mathsf{DPMarginal}}$ computes the utility scores using $\mathcal{F}_{\mathsf{Arith}}.\mathsf{abs}$ (ABS), sampling noise (Noise), and searching for the maximum noisy score (Argmax). Now, the cost of computing absolute value becomes an efficiency bottleneck. We note that this is an unavoidable cost under the framework of AIM because it needs to get all the ℓ_1 distances to support the later selection step. Under the centralized setting, computing all the utility scores also incurs $O(m^2u^2)$ computation cost. We also report the local computation time that generates the synthetic dataset given the released DP marginal. The local generation, comprising 35 iterations, is completed in 7.19 minutes.

Utility comparison with centralized SDG. Our protocol replaces all continuous Gaussian noise in the centralized setting with discrete Gaussian noise and uses the report-noisy max with Geometric noise instead of the exponential mechanism. To investigate the influence on utility, we compare the utility of our synthesis dataset in the distributed and centralized settings. We randomly sample 39,073 records (80%) to conduct data synthesis and use 9,771 records (20%) as the test set to evaluate the downstream machine learning task. We consider two types of downstream tasks on the synthetic dataset.

- **Conditional Query:** We conduct conditional queries that simulate `Join` and `GroupBy` in SQL (fix one attribute as the key column and query the frequency of the other attribute). We report ℓ_1 error of these query results. Formally, the query error can be expressed as, $\mathbb{E}_{r\in R}\left|q_r(D^{syn}) - q_r(D^{test})\right|$, where D^{syn} is the synthetic dataset and D^{test} is the test set.

- **Machine Learning:** We use the synthetic dataset to train four machine learning models: MLP, CatBoost, XGBoost, and Random Forest. The target variable (label) is set to the last attribute of the dataset, which is a binary classification label. We report the average classification accuracy.

As shown in Table 6, under different privacy budgets ε and different downstream tasks, our synthesis dataset achieves the same utility guarantee as the one synthesized in the centralized setting. This is because the discrete Gaussian mechanism has a very close utility to the continuous Gaussian mechanism [37, 38]. These evaluation results can eliminate concerns about using our protocol in practice.

Table 6: The utility of synthetic data under different methods.

Utility metric	Method	$\varepsilon = 0.1$	$\varepsilon = 0.5$	$\varepsilon = 1$	$\varepsilon = 5$	$\varepsilon = 10$
Query Error	AIM [3]	10.92%	7.06%	4.65%	1.72%	1.80%
	Ours	10.59%	7.75%	4.52%	2.08%	1.84%
ML Accuracy	Non-DP	86.47%				
	AIM [3]	82.17%	82.87%	85.07%	85.42%	85.90%
	Ours	81.66%	83.39%	84.96%	85.38%	85.77%

8 Related Work

In this section, we describe existing DP tabular data synthesis methods in both centralized and distributed settings.

Centralized DP-TDS. Centralized DP-TDS algorithms generally fall into two categories: marginal-based methods and deep learning methods. Early marginal-based approaches, such as MWEM [27], iteratively refine approximate distributions using the Multiplicative Weights framework [39]. Later, algorithms like PrivBayes, MST [40], and DPSyn [41] emerged in the NIST challenges [42, 43], showing strong performance by privately selecting and answering correlated low-dimensional marginals. Building on these, works such as PrivSyn [4], PrivMRF [33], and AIM [3] advanced the use of probabilistic graphical models, while others (e.g., FEM [44], RAP/RAP++ [45, 46]) framed synthesis as an optimization problem. In contrast, deep learning methods aim to directly learn generative models of the data. Early attempts with GANs (e.g., DP-GAN [47], PATE-GAN [48], DP-CGAN [49]) struggled to produce competitive results. More recent methods, including GEM [50] and DP-MERF [51], combine GAN with marginals or employ new loss functions.

Distributed DP-TDS. Distributed DP-TDS follows the centralized algorithms but requires efficient protocols when the dataset is split across parties. In the horizontal setting, Pereira et al. [9] use MPC to implement MWEM [27], and Maddock et al. [7] adapt AIM with a secure aggregation protocol, though the iterative exponential mechanism still makes it slow. In the vertical setting, the only work we found is CaPS [8], which relies on a general MPC framework and remains inefficient. We note that horizontal marginal computation reduces to secure histograms, which can now be handled efficiently with recent histogram protocols [13, 11, 12, 52].

9 Conclusion

In this work, we presented an efficient protocol for differentially private tabular data synthesis when raw data are held by multiple parties. We designed two novel primitives, DPF-based marginal computation and CDF-based noise generation. Our protocol achieves utility that matches the state-of-the-art centralized methods while improving efficiency by several orders of magnitude over the prior decentralized method, making distributed DP data synthesis practical at scale.

Acknowledgment

We thank all the anonymous reviewers for their valuable comments. This work is in part supported by NSF awards 2212746, 2044679, a Packard Fellowship, and a generous gift from the late Nikolai Mushegian. The work of Yucheng Fu and Tianhao Wang was supported by CNS-2213700 and CCF-2217071. The opinions, findings, conclusions, or recommendations in this work are those of the authors and do not reflect the views of the National Science Foundation.

Open Science

Artifact is available at https://doi.org/10.5281/zenodo.18218427. It includes the implementation of our protocol and the scripts to reproduce the main results in Section 7.

Ethical Considerations

This work introduces a protocol for differentially private tabular data synthesis in the distributed setting. Below, we provide a reflection focusing on data provenance, stakeholders, and the potential societal impact of our work.

Data Source and Legality. The public dataset we used in this study is the UCI Adult dataset [22], which is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license: `https://creativecommons.org/licenses/by/4.0/legalcode`. This dataset is licensed for sharing and adaptation for any purpose, provided proper credit is given. For the synthesis datasets used for evaluating efficiency, we generate them in the standard random library of Python. Thus, no personal or sensitive data were accessed or processed beyond what is publicly available. Moreover, our experiments do not involve human subjects or identifiable individuals.

Stakeholders and Potential Impact. The primary stakeholders of our work include (1) organizations that hold sensitive tabular data and wish to share it (e.g., hospitals, companies, or government agencies), (2) individuals whose data may appear in the original datasets, and (3) downstream users who rely on synthetic data for analysis or decision-making.

- For the data owners (1) and (2), our system enforces rigorous differential privacy (DP) guarantees. However, we note that DP offers only statistical privacy for data sharing rather than absolute protection. In particular, DP still allows for partial disclosure of population-level statistical characteristics and probability to distinguish between neighboring datasets. Such leakage may indirectly affect stakeholders if synthetic data are used to infer sensitive correlations. To address this concern, our paper provides a detailed mathematical analysis of the privacy guarantee of DP and cryptographic security parameters, and we recommend conservative privacy budgets (e.g., $\varepsilon \leq 0.1$) in highly sensitive applications to prevent the overexposure of distributional information.
- For the downstream data analyst (3), our system inevitably introduces a loss of data utility due to the randomness required to achieve DP. This trade-off between privacy and accuracy is inherent to all DP-based algorithms. We provide empirical evaluations in the paper to quantify this effect and to demonstrate that our protocol achieves comparable utility to its centralized DP data synthesis algorithms. Nevertheless, we remind users that DP-synthetic datasets are approximations of the original data, and any critical decision-making or statistical inference should consider the uncertainty introduced by the noise.

Responsible Disclosure and Dual-Use Concerns. Our algorithms and protocols are designed for legitimate data-sharing and collaborative research purposes. However, we acknowledge that synthetic data generation techniques might be misused by criminal organizations. For instance, generating datasets for deceptive or malicious purposes. To address this, during development, we set restrictions on our experiment platform (e.g., EC2 servers) that only allow our research team to access. We also explicitly commit that neither the authors nor any affiliated institutions will use or release the system for generating synthetic data that could cause societal or ethical harm (e.g., fake statistics, discriminatory data, or data supporting unlawful activities). We clearly state that after open source, we will monitor for any issues and take responsibility if they arise, working to remediate any harm. This proactive approach ensures our research remains ethical and responsible.

In summary, our work aims to advance privacy-preserving data analysis and does not introduce new privacy or ethical risks beyond existing differentially private data synthesis and cryptography frameworks.

References

[1] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "Privbayes: Private data release via bayesian networks," *ACM Transactions on Database Systems (TODS)*, vol. 42, no. 4, pp. 1–41, 2017.

[2] R. McKenna, D. Sheldon, and G. Miklau, "Graphical-model based estimation and inference for differential privacy," in *International Conference on Machine Learning*. PMLR, 2019, pp. 4435–4444.

[3] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "Aim: an adaptive and iterative mechanism for differentially private synthetic data," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, 2022.

[4] Z. Zhang, T. Wang, N. Li, J. Honorio, M. Backes, S. He, J. Chen, and Y. Zhang, "Privsyn: Differentially private data synthesis," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 929–946. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/zhang-zhikun

[5] G. Cormode, S. Maddock, E. Ullah, and S. Gade, "Synthetic tabular data: Methods, attacks and defenses," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 5989–5998.

[6] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS'07)*. IEEE, 2007, pp. 94–103.

[7] S. Maddock, G. Cormode, and C. Maple, "Flaim: Aim-based synthetic data generation in the federated setting," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 2165–2176.

[8] S. Pentyala, M. Pereira, and M. De Cock, "Caps: Collaborative and private synthetic data generation from distributed sources," in *Forty-first International Conference on Machine Learning*, 2024.

[9] M. Pereira, S. Pentyala, A. Nascimento, R. T. d. Sousa Jr, and M. De Cock, "Secure multiparty computation for synthetic data generation from distributed data," *arXiv preprint arXiv:2210.07332*, 2022.

[10] S. Zhang, H. Sun, K. Knopf, S. Mohapatra, W. Pang, C. Wang, Y. Wang, M. Shafieinejad, D. Emerson, and X. He, "Feddpsyn: Federated tabular data synthesis with computational differential privacy."

[11] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *14th USENIX symposium on networked systems design and implementation (NSDI 17)*, 2017, pp. 259–282.

[12] E. Anderson, M. Chase, F. B. Durak, K. Laine, and C. Weng, "Precio: Private aggregate measurement via oblivious shuffling," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 1819–1833.

[13] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, "Lightweight techniques for private heavy hitters," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 762–776.

[14] G. Asharov, K. Hamada, R. Kikuchi, A. Nof, B. Pinkas, and J. Tomida, "Secure statistical analysis on multiple datasets: Join and group-by," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 3298–3312.

[15] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor, "Our data, ourselves: Privacy via distributed noise generation," in *Advances in Cryptology-EUROCRYPT 2006: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28-June 1, 2006. Proceedings 25*. Springer, 2006, pp. 486–503.

[16] C. Wei, R. Yu, Y. Fan, W. Chen, and T. Wang, "Securely sampling discrete gaussian noise for multi-party differential privacy," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023.

[17] J. Champion, A. Shelat, and J. Ullman, "Securely sampling biased coins with applications to differential privacy," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 603–614.

[18] Y. Fu and T. Wang, "Benchmarking secure sampling protocols for differential privacy," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.

[19] F. Eigner, A. Kate, M. Maffei, F. Pampaloni, and I. Pryvalov, "Differentially private data aggregation with optimal utility," in *Proceedings of the 30th Annual Computer Security Applications Conference*, 2014, pp. 316–325.

[20] H. Keller, H. Möllering, T. Schneider, O. Tkachenko, and L. Zhao, "Secure noise sampling for dp in mpc with finite precision," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, 2024, pp. 1–12.

[21] G. E. Box and M. E. Muller, "A note on the generation of random normal deviates," *The annals of mathematical statistics*, vol. 29, no. 2, pp. 610–611, 1958.

[22] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: https://doi.org/10.24432/C5XW20.

[23] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Tfhe: fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.

[24] A. Agarwal, E. Boyle, N. Gilboa, Y. Ishai, M. Kelkar, and Y. Ma, "Compressing unit-vector correlations via sparse pseudorandom generators," in *Annual International Cryptology Conference*. Springer, 2024, pp. 346–383.

[25] G. E. Blelloch, "Prefix sums and their applications," 1990.

[26] G. Asharov, K. Hamada, D. Ikarashi, R. Kikuchi, A. Nof, B. Pinkas, K. Takahashi, and J. Tomida, "Efficient secure three-party sorting with applications to data analysis and heavy hitters," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 125–138.

[27] M. Hardt, K. Ligett, and F. McSherry, "A simple and practical algorithm for differentially private data release," *Advances in neural information processing systems*, vol. 25, 2012.

[28] M. Aliasgari, M. Blanton, Y. Zhang, and A. Steele, "Secure computation on floating point numbers," *Cryptology ePrint Archive*, 2012.

[29] I. Mironov, O. Pandey, O. Reingold, and S. Vadhan, "Computational differential privacy," in *Annual International Cryptology Conference*. Springer, 2009, pp. 126–142.

[30] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*, R. Safavi-Naini and R. Canetti, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 643–662.

[31] M. Keller, "Mp-spdz: A versatile framework for multi-party computation," in *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, 2020, pp. 1575–1590.

[32] O. Goldreich, S. Goldwasser, and S. Micali, "How to construct random functions," *Journal of the ACM (JACM)*, vol. 33, no. 4, pp. 792–807, 1986.

[33] K. Cai, X. Lei, J. Wei, and X. Xiao, "Data synthesis via differentially private markov random fields," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2190–2202, 2021.

[34] D. Demmler, T. Schneider, and M. Zohner, "Aby-a framework for efficient mixed-protocol secure two-party computation." in *NDSS*, 2015.

[35] G. Ganev, M. S. Muthu Selva Annamalai, S. Mahiou, and E. De Cristofaro, "The importance of being discrete: Measuring the impact of discretization in end-to-end differentially private synthetic data," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 3236–3250.

[36] K. Chen, X. Li, C. Gong, R. McKenna, and T. Wang, "Benchmarking differentially private tabular data synthesis," *arXiv preprint arXiv:2504.14061*, 2025.

[37] P. Kairouz, Z. Liu, and T. Steinke, "The distributed discrete gaussian mechanism for federated learning with secure aggregation," in *International Conference on Machine Learning*. PMLR, 2021, pp. 5201–5212.

[38] C. L. Canonne, G. Kamath, and T. Steinke, "The discrete gaussian for differential privacy," *Advances in Neural Information Processing Systems*, vol. 33, pp. 15 676–15 688, 2020.

[39] M. Hardt and G. N. Rothblum, "A multiplicative weights mechanism for privacy-preserving data analysis," in *2010 IEEE 51st annual symposium on foundations of computer science*. IEEE, 2010, pp. 61–70.

[40] R. Mckenna, G. Miklau, and D. Sheldon, "Winning the nist contest: A scalable and general approach to differentially private synthetic data," *Journal of Privacy and Confidentiality*, vol. 11, p. 3, 2021.

[41] N. Li, Z. Zhang, and T. Wang, "Dpsyn: Experiences in the nist differential privacy data synthesis challenges," *arXiv preprint arXiv:2106.12949*, 2021.

[42] NIST, "2018 differential privacy synthetic data challenge," Available at https://www.nist.gov/ctl/pscr/open-innovation-prize-challenges/past-prize-challenges/2018-differential-privacy-synthetic, 2019.

[43] ——, "2020 differential privacy temporal map challenge," Available at https://www.nist.gov/ctl/pscr/open-innovation-prize-challenges/current-and-upcoming-prize-challenges/2020-differential, 2021.

[44] G. Vietri, G. Tian, M. Bun, T. Steinke, and S. Wu, "New oracle-efficient algorithms for private synthetic data release," in *International Conference on Machine Learning*. PMLR, 2020, pp. 9765–9774.

[45] S. Aydore, W. Brown, M. Kearns, K. Kenthapadi, L. Melis, A. Roth, and A. A. Siva, "Differentially private query release through adaptive projection," in *International Conference on Machine Learning*. PMLR, 2021, pp. 457–467.

[46] G. Vietri, C. Archambeau, S. Aydore, W. Brown, M. Kearns, A. Roth, A. Siva, S. Tang, and S. Z. Wu, "Private synthetic data for multitask learning and marginal queries," *Advances in Neural Information Processing Systems*, vol. 35, pp. 18 282–18 295, 2022.

[47] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou, "Differentially private generative adversarial network," *arXiv preprint arXiv:1802.06739*, 2018.

[48] J. Jordon, J. Yoon, and M. Van Der Schaar, "Pate-gan: Generating synthetic data with differential privacy guarantees," in *International conference on learning representations*, 2018.

[49] R. Torkzadehmahani, P. Kairouz, and B. Paten, "Dp-cgan: Differentially private synthetic data and label generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019, pp. 0–0.

[50] T. Liu, G. Vietri, and S. Z. Wu, "Iterative methods for private synthetic data: Unifying framework and new methods," *Advances in Neural Information Processing Systems*, vol. 34, pp. 690–702, 2021.

[51] F. Harder, K. Adamczewski, and M. Park, "Dp-merf: Differentially private mean embeddings with randomfeatures for practical privacy-preserving data generation," in *International conference on artificial intelligence and statistics*. PMLR, 2021, pp. 1819–1827.

[52] D. Keeler, C. Komlo, E. Lepert, S. Veitch, and X. He, "Dprio: Efficient differential privacy with high utility for prio," *Proceedings on Privacy Enhancing Technologies*, no. 3, 2023.

[53] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Theory of cryptography conference*. Springer, 2016, pp. 635–658.

Appendix

A Distribution Definitions

Definition A.1 (Discrete Gaussian Distribution)**.** *The probability mass function (PMF) of the discrete Gaussian distribution $\mathcal{N}(\sigma)$ with scale parameter $\sigma > 0$ is defined as*

$$f_{\mathcal{N}(\sigma)}(x) = \frac{1}{c} \cdot e^{-\frac{x^2}{2\sigma^2}}, \qquad x \in \mathbb{Z},$$

where $c = \sum_{x\in\mathbb{Z}} e^{-\frac{x^2}{2\sigma^2}}$ is the normalization constant.

In our definition, we replace the success probability α with a scale parameter σ, where $\alpha = e^{-1/\sigma}$. This makes the distribution decay exponentially in x at a rate controlled by σ, aligning with the role of σ as a noise scale in DP.

Definition A.2 (Geometric Distribution)**.** *The probability mass function (PMF) of the Geometric distribution $\mathcal{G}(\sigma)$ with scale parameter $\sigma > 0$ is defined as*

$$f_{\mathcal{G}(\sigma)}(k) = (1-\alpha)^{k-1}\alpha, \qquad k \in \mathbb{N}^+,$$

where $\alpha = e^{-1/\sigma} \in (0,1)$ is the success probability.

Definition A.3 (Statistical Distance)**.** *The statistical distance between distributions $\mathcal{V}$ and $\mathcal{W}$ over a finite set $\mathbb{F}$ is defined as $\mathcal{D}(\mathcal{V},\mathcal{W}) = \frac{1}{2}\sum_{x\in\mathbb{F}} |f_{\mathcal{V}}(x) - f_{\mathcal{W}}(x)|$.*

B Zero-Concentrated Differential Privacy

Definition B.1 (Zero-Concentrated Differential Privacy (zCDP) [53])**.** *A randomized mechanism $M : \mathbb{F} \to \mathbb{O}$ is said to satisfy ρ-zCDP if for all pairs of neighboring datasets $D, D_s \in \mathbb{F}$ (differing in one entry), and for all $\alpha \in (1,\infty)$,*

$$D_\alpha(M(D) \parallel M(D_s)) \le \rho \cdot \alpha,$$

where $D_\alpha(P \parallel Q)$ denotes the Rényi divergence of order α between distributions P and Q, defined as

$$D_\alpha(P \parallel Q) = \frac{1}{\alpha - 1} \log \mathbb{E}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}\right)^\alpha\right].$$

The composition property of zCDP [53] is defined as,

Proposition B.2 (Sequential Composition of zCDP Mechanisms)**.** *If two randomized mechanisms M_1 and M_2 satisfy ρ_1-zCDP and ρ_2-zCDP respectively, their sequential composition $M = (M_1, M_2)$ satisfies $(\rho_1 + \rho_2)$-zCDP.*

The following proposition states the zCDP guarantee of the discrete Gaussian mechanism.

Proposition B.3 (Discrete Gaussian Mechanism [38])**.** *Let $f : \mathcal{D} \to \mathbb{R}^k$ be a query and let neighboring datasets D, D' differ in one record. Define the ℓ_2 sensitivity of f by*

$$\Delta_2 \triangleq \max_{D\sim D'} \|f(D) - f(D')\|_2.$$

The discrete Gaussian mechanism that releases $\mathcal{M}(D) = f(D) + \mathcal{N}(\sigma^2\mathbb{I})$ satisfies $\left(\frac{\Delta_2^2}{2\sigma^2}\right)$-zCDP.

The relationship between zCDP and DP is stated as follows.

Proposition B.4 (zCDP to DP [38])**.** *If a mechanism $\mathcal{M}$ satisfies ρ-zCDP, then it also satisfies (ε,δ)-DP for all $\varepsilon \ge 0$, where $\delta = \min_{\alpha>1} \exp\Big((\alpha-1)(\alpha\rho - \varepsilon)\Big) \cdot \frac{1}{\alpha-1}\left(1 - \frac{1}{\alpha}\right)^\alpha$.*

Proposition B.5 (DP to zCDP [53])**.** *If a mechanism $\mathcal{M}$ satisfies ε-DP, then $\mathcal{M}$ also satisfies $\left(\frac{\varepsilon^2}{2}\right)$-zCDP.*

C Details of Sort-count

We tried to adapt the joint histogram computation protocol in [26] to compute two-way marginals, as shown in Figure 10. Using the three-party sorting protocol, the total communication cost of Sort-count becomes $O(m^2(n+u^2)\log u)$.

Protocol $\Pi_{\mathsf{Sort-count}}$

Parameters. Number of records n, number of attributes m.
Input. Secret-shared columns $\{\langle \mathbf{Col} \rangle_c\}_{c\in[m]}$.
Extract Marginals.

1. For $1 \le a < b \le m$, compute two-way marginal $\langle \mathbf{M}_{a,b} \rangle$:
   - For $1 \le i \le n$: $\langle \mathbf{Col}_{a,b}[i] \rangle \leftarrow (\langle \mathbf{Col}_a[i] \rangle, \langle \mathbf{Col}_b[i] \rangle)$.
   - For $1 \le i \le u_a$ and $1 \le j \le u_b$, append (i, j):
   $$\langle \mathbf{Col}_{a,b} \rangle \leftarrow \langle \mathbf{Col}_{a,b} \rangle || (\langle i \rangle, \langle j \rangle).$$
   - Sort $\mathbf{Col}_{a,b}$ using $\langle \mathbf{Col}'_a \rangle$ as the primary key and $\langle \mathbf{Col}'_b \rangle$ as the secondary key:
   $$\langle \mathbf{Col}_{a,b} \rangle \leftarrow \mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{Col}_{a,b} \rangle, \langle \mathbf{Col}'_b \rangle, \lceil \log u_b \rceil).$$
   $$\langle \mathbf{Col}_{a,b} \rangle \leftarrow \mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{Col}_{a,b} \rangle, \langle \mathbf{Col}'_a \rangle, \lceil \log u_a \rceil).$$
   - Let $\langle \mathbf{b}[i] \rangle \leftarrow \langle 0 \rangle$. For $2 \le i \le n + u_a u_b$:
   $$\langle \mathbf{b}[i] \rangle \leftarrow (\langle \mathbf{Col}_{a,b}[i-1] \rangle == \langle \mathbf{Col}_{a,b}[i] \rangle).$$
   - For $1 \le i \le n + u_a u_b$: $\langle \mathbf{x}[i] \rangle = \langle i \rangle$.
   - Compact $\langle \mathbf{x} \rangle$ using $\langle \mathbf{b} \rangle$ as the key:
   $$\langle \mathbf{x} \rangle \leftarrow \mathcal{F}_{\mathsf{Sort}}(\langle \mathbf{x} \rangle, \langle \mathbf{b} \rangle, 1).$$
   - Let $\langle \mathbf{M}_{a,b}[1] \rangle \leftarrow \langle \mathbf{x}[1] \rangle$. For $2 \le i \le u_a u_b$:
   $$\langle \mathbf{M}_{a,b}[i] \rangle \leftarrow \langle \mathbf{x}[i] \rangle - \langle \mathbf{x}[i-1] \rangle - 1.$$
2. For $1 \le a \le m$, compute one-way marginals:
   $$\langle \mathbf{M}_a \rangle \leftarrow \sum_{i \in [u_b]} \langle \mathbf{M}_{a,i} \rangle.$$

Outputs. Secret shares $\{\langle \mathbf{M}_a \rangle\}_{1\le a\le m}$, $\{\langle \mathbf{M}_{a,b} \rangle\}_{1\le a<b\le m}$.

Figure 10: The protocol $\Pi_{\mathsf{Sort-count}}$ adapted from [14] that computes all the one-way and two-way marginals.

D Configuration of Truncation and CDF Precision Parameters for $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$

Let $\mathcal{N}^d_{s,p}(\sigma)$ be the distribution of $\langle \eta \rangle$. We describe how to derive the truncation parameter s and CDF precision parameter p such that the d targeted samples output by the protocol (denoted by $\mathcal{N}^d_{s,p}(\sigma)$) have at most $2^{-\lambda}$ statistical distance to those from the "perfect" distribution $\mathcal{N}^d(\sigma)$. By triangle inequality $\mathcal{D}(\mathcal{N}^d_{s,p}(\sigma), \mathcal{N}^d(\sigma)) \le \mathcal{D}(\mathcal{N}^d_s(\sigma), \mathcal{N}^d(\sigma)) + \mathcal{D}(\mathcal{N}^d_{s,p}(\sigma), \mathcal{N}^d_s(\sigma))$, where $\mathcal{D}(\mathcal{N}^d_s(\sigma), \mathcal{N}^d(\sigma)) = 2^{-\lambda}/2$ is defined as the statistical distance caused by the truncation of samples (related to s) and $\mathcal{D}(\mathcal{N}^d_{s,p}(\sigma), \mathcal{N}^d_s(\sigma)) = 2^{-\lambda}/2$ is defined as the statistical distance caused by the precision of representing CDF (related to p). We derive s and p as follows:

- **Truncation (s).** Given a standard deviation σ, the statistical distance between $\mathcal{N}(\sigma)$ and $\mathcal{N}_s(\sigma)$ is bounded by $2e^{-\frac{s^2}{2\sigma^2}}$ [37]. Therefore, we have,
$$\begin{aligned}\mathcal{D}(\mathcal{N}^d(\sigma), \mathcal{N}^d_s(\sigma)) &\le d \cdot \mathcal{D}(\mathcal{N}(\sigma), \mathcal{N}_s(\sigma)) \\ &= 2de^{-\frac{s^2}{2\sigma^2}} = 2^{-\lambda}/2.\end{aligned}$$
By deriving s, the $\mathsf{parameter\_solver}(d, \sigma, \lambda)$ configures s as the smallest integer such that,
$$s \ge \sigma \cdot \sqrt{2\ln 2(\lambda + 2 + \log d)} \tag{1}$$
- **CDF Precision (p).** We begin with the statistical distance between $\mathcal{N}_s(\sigma)$ and $\mathcal{N}_{s,p}(\sigma)$. For each $x \in [-s, s]$, we have,
$$\begin{aligned}&\mathcal{D}(\mathcal{N}_s(\sigma), \mathcal{N}_{s,p}((\sigma)) \\ =&\frac{1}{2}\sum_{-s\le x\le s}\left|f_{\mathcal{N}_s(\sigma)}(x) - f_{\mathcal{N}_{s,p}(\sigma)}(x)\right|\end{aligned}$$
where the CDF $f_{\mathcal{N}_{s,p}(\sigma)}(x)$ is represented by a p-bit fixed-point number. Thus, we have $\left|f_{\mathcal{N}_s(\sigma)}(x) - f_{\mathcal{N}_{s,p}(\sigma)}(x)\right| = 2^{-p}/2$ for $x \in [-s, s]$. Therefore, the statistical distance is,
$$\begin{aligned}\mathcal{D}(\mathcal{N}^d_s(\sigma), \mathcal{N}^d_{s,p}(\sigma)) &\le d \cdot \mathcal{D}(\mathcal{N}_s(\sigma), \mathcal{N}_{s,p}(\sigma)) \\ &\le \frac{d}{2}\sum_{-s\le x\le s}\left|f_{\mathcal{N}_s(\sigma)}(x) - f_{\mathcal{N}_{s,p}(\sigma)}(x)\right| \\ &\le \frac{d}{2}(2s+1)(2^{-p}/2) = 2^{-\lambda}/2.\end{aligned}$$
By deriving p, the $\mathsf{parameter\_solver}(d, \sigma, \lambda)$ configures p as the smallest integer such that,
$$p \ge \lambda + \log d + \log(2s+1) + \ln 2 + 1 \tag{2}$$

E Parallel Segmented Scan

Inputs of our segmented scan are two secret shared vectors $\langle \mathbf{t}' \rangle$ and $\langle \mathbf{b} \rangle$, where $\langle \mathbf{b} \rangle$ contains $\langle 0 \rangle$ or $\langle 1 \rangle$ and $\langle \mathbf{t}'[i] \rangle = \langle \perp \rangle$ if $\langle \mathbf{b}[i] \rangle = \langle 0 \rangle$. We aim to use the segmented scan to change every $\langle \mathbf{b}[i] \rangle$ with flag $\langle \mathbf{t}'[i] \rangle = \langle 0 \rangle$ into the closest value $\langle \mathbf{t}'[j] \rangle$, where $j < i$ and $\mathbf{t}[j] = 0$. In Figure 2, we adapt the exclusive prefix sum in [25] and present it without the notion of secret sharing. We refer readers to [25] for more details

We set $\mathbf{t}'[i] = \mathbf{t}'[i] \cdot \mathbf{b}[i]$ for all the elements in the payload $\mathbf{t}'$ at the beginning. This operation ensures that all the elements with flag $\mathbf{b}[i] = 0$ will also be converted into 0. Then, treating $\mathbf{t}'$ with exclusive prefix sum can give us the desired output with valid values propagated downwards. Denoted by n the length of $\mathbf{t}'$. When instantiated with secret sharing, this algorithm only incurs $O(\log n)$ rounds and $O(n)$ communication. The reason is that we can compute $\mathbf{t}'[i] = \mathbf{t}'[i] \cdot \mathbf{b}[i]$ with n secure multiplication in parallel. Then, in the up sweep, the

Algorithm 2 segmented_scan

Input: Payload vector $\mathbf{t}'$, flag vector $\mathbf{b}$.
Output: $\mathbf{t}'$ with valid elements propagated downwards.

$\mathbf{t}'[i] \leftarrow \mathbf{t}'[i] \cdot \mathbf{b}[i], \forall i \in [n]$.
$\hat{\mathbf{b}} \leftarrow \mathbf{b}$
for $i \leftarrow 0$ to $h-1$ **do** ▷ Up sweep
  for $j \leftarrow 0$ to $n-1$ by 2^{i+1} in parallel **do**
    $\ell \leftarrow j + 2^i - 1, \quad r \leftarrow j + 2^{i+1} - 1$
    if $\mathbf{b}[r] = 0$ **then**
      $\mathbf{t}'[r] \leftarrow \mathbf{t}'[r] + \mathbf{t}'[\ell]$
    end if
    $\mathbf{b}[r] \leftarrow \mathbf{b}[r] \vee \mathbf{b}[\ell]$
  end for
end for
$\mathbf{t}'[n-1] \leftarrow 0$
for $i \leftarrow h-1$ down to 0 **do** ▷ Down sweep
  for $j \leftarrow 0$ to $n-1$ by 2^{i+1} in parallel **do**
    $\ell \leftarrow j + 2^i - 1, \quad r \leftarrow j + 2^{i+1} - 1, \quad \text{tmp} \leftarrow \mathbf{t}'[\ell]$
    $\mathbf{t}'[\ell] \leftarrow \mathbf{t}'[r]$
    if $\hat{\mathbf{b}}[j + 2^i] = 1$ **then**
      $\mathbf{t}'[r] \leftarrow 0$
    else if $\mathbf{b}[\ell] = 1$ **then**
      $\mathbf{t}'[r] \leftarrow \text{tmp}$
    else
      $\mathbf{t}'[r] \leftarrow \mathbf{t}'[r] + \text{tmp}$
    end if
    $\mathbf{b}[\ell] \leftarrow 0$
  end for
end for
return $\mathbf{t}'$

computation in each layer of the tree can be conducted in parallel, and the total number of bitwise OR is $O(n)$. Therefore, the up sweep can be done in $O(n)$ communication and $O(\log n)$ rounds. The situation is the same for the down sweep. As a result, the entire algorithm implemented with MPC has $O(n)$ communication and $O(\log n)$ rounds.

F From Semi-honest to Malicious Security

While we consider a semi-honest adversary corrupting a single server in this work, it is possible to upgrade our approach to a malicious model. We highlight the key adaptations below.

1. Multi-party DPF [24]. By applying the sketching technique in [13], we can achieve security against both the malicious clients and servers.

2. Three-party sorting [26]. The three-party sorting protocol also has its malicious version, as discussed in [26]. In a nutshell, it adds information-theoretic message authentication codes (MACs) to every secret share throughout the computation and performs a lightweight verification step before any revealing or resharing operation to ensure no additive errors were introduced.

3. Other MPC sub-protocols. Other standard MPC sub-protocols, such as comparison, multiplication, and absolute value computation, have efficient maliciously-secure variants in existing frameworks, such as MP-SPDZ [31].

G Deferred Proofs

G.1 Proof of Lemma 1

Lemma 2 (Lemma 1, restated)**.** $\mathcal{F}_{\mathsf{DPMarginal}}$ *satisfies* $(\rho_1 + \rho_2)$*-zero concentrated differential privacy.*

Proof. The $\mathcal{F}_{\mathsf{DPMarginal}}$ performs two DP mechanisms:

1. **Selection.** We apply report-noisy-max using geometric noise with parameter $\sigma_1 = 2\sqrt{\frac{2}{\rho_1}}$. In Definition 2.2, replacing a record makes $q_{a,b} = \|\mathbf{M}_{a,b} - \mathbf{M}'_{a,b}\|_1$ change at most 2, thus its sensitivity Δ is 2. Since report noisy max with geometric noise $\mathcal{G}\left(\frac{2\Delta}{\varepsilon}\right)$ satisfies ε-DP [17], it satisfies $\sqrt{2\rho_1}$-DP and satisfies ρ_1-zCDP (Proposition B.5).

2. **Adding noise.** We add discrete Gaussian noise with standard deviation $\sigma_2 = 2\sqrt{\frac{2}{\rho_2}}$ to the selected two-way marginal. Since the ℓ_2 sensitivity of each marginal entry is $\Delta_2^2 = 4$, this implies ρ_2-zCDP (Proposition B.3).

By the sequential composition of zCDP, $\mathcal{F}_{\mathsf{DPMarginal}}$ satisfies $(\rho_1 + \rho_2)$-zCDP. □

G.2 Proof of Theorem 5.1

Theorem G.1 (Theorem 5.1, restated)**.** *Protocol* $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ *securely emulates* $\mathcal{F}^{\mathcal{N}}_{\mathsf{Noise}}$ *in* $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Sort}}, \mathcal{F}_{\mathsf{Shuffle}})$*-hybrid model against semi-honest adversary.*

Proof. **Correctness.** The correctness relies on the parameter configuration derived in Appendix D. Specifically, we set the truncation parameter s and CDF precision p (Equation 1 and Equation 2) such that the statistical distance introduced by domain truncation and fixed-point representation is each bounded by $2^{-\lambda/2}$. By the triangle inequality, the total statistical distance between the protocol output and the ideal distribution $\mathcal{N}^d(\sigma)$ is at most $2^{-\lambda}$. Thus, the output is statistically indistinguishable from the ideal functionality.
Security. Security follows from the $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Sort}}, \mathcal{F}_{\mathsf{Shuffle}})$-hybrid model. The protocol performs only local operations on secret shares and invokes secure sub-functionalities. A simulator can trivially emulate the view by generating random shares for the outputs of these functionalities. □

G.3 Proof of Theorem 5.2

Theorem G.2 (Theorem 5.2, restated)**.** *Let d be the number of samples, λ be the statistical security parameter, and σ be the scale parameter. Denote $\phi = \lambda + \log d$. Then $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ has $O\left(d\phi + \sigma\phi^{3/2}\right)$ communication complexity and $O\left(\log(d+\sigma)+\phi\right)$ round complexity.*

Proof. The main cost of protocol $\Pi^{\mathcal{N}}_{\mathsf{Noise}}$ includes:

1. We call $\mathcal{F}_{\mathsf{Sort}}$ to sort $[\![\mathbf{t}']\!]$ using $[\![\mathbf{z}']\!]$ as the key. Since we use p bits to represent the elements in the key $[\![\mathbf{z}']\!]$ and the length of the payload (key) is $d+2s+1$, replacing this ideal functionality with a radix sorting protocol incurs $O\left((d+s)p\right)$ communication and $O(p)$ rounds.

2. In the propagation, we conduct a segmented scan over a vector $[\![\mathbf{t}']\!]$ of length $d+2s+1$, which has $O(d+s)$ communication and $O(\log(d+s))$ round complexity.

3. To extract the samples, we sort the vector $[\![\mathbf{y}']\!]$ of length $d+2s+1$ to compact the results. However, since the key vector $[\![\mathbf{b}]\!]$ is binary, this sorting only needs $O(d+s)$ communication and constant rounds. Finally, we shuffle the first d elements in $[\![\mathbf{y}']\!]$, which requires $O(d)$ communication and constant rounds.

Altogether, the communication cost is $O\left((d+s)p\right)$. By substituting $s = O(\sigma\sqrt{\phi})$ and $p = O(\phi)$, we obtain

$$O\left((d+\sigma\sqrt{\phi})\cdot\phi\right) = O\left(d\phi+\sigma\phi^{3/2}\right).$$

Also, the round complexity becomes: $O(\log d + s + p) = O(\log(d+\sigma\sqrt{\phi})+\phi) = O\left(\log(d+\sigma)+\phi\right)$. □

G.4 Proof of Theorem 6.1

Theorem G.3 (Theorem 6.1, restated)**.** $\Pi_{\mathsf{DPMarginal}}$ *securely emulates $\mathcal{F}_{\mathsf{DPMarginal}}$ in $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Noise}})$-hybrid model against semi-honest adversary controlling one single server.*

Proof. **Correctness.** The correctness of DPF full evaluation ensures that for each $\mathbf{Col}_l[i]$, we can obtain the secret-shared one-hot encoding $\mathbf{v}_l[i]$ with 1 in the $(\mathbf{Col}_l[i])$-th entry. After that, servers conduct homomorphic computation on these secret-shared one-hot encodings, which ensures the correctness of any two-way marginal $\langle \mathbf{M}_{a,b} \rangle$. The correctness of Select and Measure steps relies on $\mathcal{F}_{\mathsf{Noise}}$, which ensures the final selected two-way marginal index (a^*, b^*) and the corresponding noisy two-way marginal $\tilde{\mathbf{M}}_{a^*,b^*}$ are indistinguishable from the corresponding outputs of $\mathcal{F}_{\mathsf{DPMarginal}}$.

Simulator. Without loss of generality, let $\mathcal{A}$ be an adversary controlling server P_1. We describe the ideal-world process with the help of a simulator $\mathcal{S}$ that interacts with $\mathcal{A}$. On the first call of $\mathcal{F}_{\mathsf{DPMarginal}}$, for each $c \in [m]$ and $i \in [n]$, the simulator $\mathcal{S}$ calls the simulator of three-party DPF and obtains three DPF keys $(k^1_{c,i}, k^2_{c,i}, k^3_{c,i})$ and hands $k^1_{c,i}$ to $\mathcal{A}$. Then, $\mathcal{A}$ performs the local operations on these keys and obtains the server P_1's shares of all the two-way marginals $\{\langle \mathbf{M}_{a,b} \rangle_1\}_{a,b\in\binom{m}{2}}$. In the later steps, the parties do not communicate with each other in the $(\mathcal{F}_{\mathsf{Arith}}, \mathcal{F}_{\mathsf{Noise}})$-hybrid model. At the end of the protocol, for any output message x being constructed, $\mathcal{S}$ takes the corresponding output from $\mathcal{F}_{\mathsf{DPMarginal}}$ and the share $\langle x \rangle_1$ from P_1 to compute the other two shares $\langle x \rangle_2$, $\langle x \rangle_3$. Then, it sends $\langle x \rangle_2$, $\langle x \rangle_3$ to $\mathcal{A}$ to simulate the real-world message from the other two servers.

Indistinguishability. Based on the security of DPF, all the keys $\mathcal{A}$ receives from the simulator of DPF are computationally indistinguishable from those keys in the real world. As for the output phase, in both the ideal world and real world, $\mathcal{A}$ has the shares $\langle x \rangle_1$, $\langle x \rangle_2$, $\langle x \rangle_3$ for any reconstructed message x. Because x has indistinguishable distributions in both the ideal world and real world, it is immediate that the view of the adversary during the simulation is computationally indistinguishable from its view in the real-world execution. □

Algorithm 3 data_preprocessing

Input: Column **Col**; privacy budget ρ; domain $\mathcal{U}$ of size u, number of records n.
Output: Encoded column $\hat{\mathbf{Col}}$ and compressed domain $\hat{\mathcal{U}}$.

Set $\sigma \leftarrow 2\sqrt{\frac{2}{\rho}}$, rare $\leftarrow \emptyset$.
$\mathbf{M} \leftarrow$ 1-way marginal of column **Col**.
$\tilde{\mathbf{M}} \leftarrow \mathbf{M} + \mathcal{N}(\sigma)$. ▷ Discrete Gaussian noise
for $i \leftarrow 1$ to u **do**
  if $\tilde{\mathbf{M}}[i] < 3\sigma$ **then**
    Add i to rare and remove i from $\mathcal{U}$.
  end if
end for
$\hat{\mathcal{U}} \leftarrow \mathcal{U} \cup \{\perp\}$, $\hat{u} \leftarrow u - |\text{rare}| + 1$.
for $i \leftarrow 1$ to n **do**
  if $\mathbf{Col}[i] \in$ rare **then**
    $\mathbf{Col}[i] \leftarrow \hat{u}$.
  end if
end for
return $\hat{\mathbf{Col}}$, $\hat{\mathcal{U}}$.

H Details of Domain Compression Algorithm

A description of the domain compression is in Algorithm 3. Because **Col** is input to the protocol $\Pi_{\mathsf{DPMarginal}}$ later and will not be directly published, it does not incur additional privacy leakage.